Chapter 15 — Scaling Databases: Partitioning, Sharding, Replication and Recovery

A single PostgreSQL server goes a long way: one machine with plenty of **RAM** (the fast working memory a computer loses when powered off), fast **SSDs** (solid-state drives — disks with no moving parts, much faster than spinning hard drives) and good indexes (Chapter 12) can serve thousands of queries per second. Sooner or later, though, the data outgrows one disk, the traffic outgrows one **CPU** (the processor that executes instructions), or the business can't accept that one broken machine takes the whole product offline. This chapter is about what databases do when **one box is no longer enough**.

Two ideas carry most of the weight:

- **Partitioning / sharding**: split the data so each piece is smaller (more capacity, faster scans).
- **Replication**: copy the data so there is more than one of each piece (availability, read scaling, disaster recovery).

Then come the problems they create: copies disagree (consistency models, CAP), machines must agree (consensus, leader election), and failures must not lose committed data (logging, checkpoints, two-phase commit, failover, backups).

What you'll learn

Section	Topic	Runnable part
15.1	What a distributed database is, scale up vs scale out	—
15.2	Partitioning: horizontal/vertical, range/list/hash/composite/round-robin	Postgres declarative partitioning + pruning with `EXPLAIN`
15.3	Sharding, shard keys, consistent hashing	Python: keys moved when a shard is added
15.4	Distributed query processing	`postgres_fdw`: predicate/aggregate push-down
15.5	Replication: types, sync vs async, single/multi/leaderless	Python: quorum reads/writes
15.6	Consistency models	—
15.7	CAP and PACELC	—
15.8	Consensus (Paxos, Raft, PBFT, PoW) and leader election	Python: Bully vs Ring message counts
15.9	Fault tolerance: WAL, redo/undo, LSNs, checkpoints, 2PC/3PC	Postgres WAL/LSN demo, Python recovery + 2PC
15.10	High availability and failover	—
15.11	Data resilience: redundancy, mirroring, backups	—

> **Going deeper.** This chapter explains each idea once, at interview depth. For book-level depth see your DDIA notes — Replication, Partitioning, Trouble with distributed systems, Consistency and consensus — and for the design-interview pattern view the system-design-patterns notes: Storing data (primary-replica, sharding, consistent hashing, WAL), Consistency (2PC, saga, quorum, vector clocks), Scaling, Reliability.

All SQL files are in `code/` and run on the book's private PostgreSQL 14 server (`bash tools_run_sql.sh sql '15-*'`). `00_schema.sql` creates 12 customers in three regions (`NA`, `EU`, `APAC`) and 3,000 orders

spread over 2022–2024. Python simulations run with the books' venv (`.venv-books/bin/python code/<file>.py`).

15.1 Distributed databases: the big picture

What it is

A **distributed database** stores its data on several machines (**nodes**), possibly in different **data centres** (buildings full of servers, power and network gear), but presents itself to applications as *one* database. Think of a library with branches across a city: you search one catalogue and borrow from whichever branch has the book, without knowing which building holds which shelf.

Term / characteristic	Meaning
Node	One server (physical or virtual) running a database process. **Cluster** = the group of nodes forming the database.
Transparency	The client doesn't need to know *where* a row is stored (**location transparency**) or *how many copies* exist (**replication transparency**).
Fault tolerance	The system keeps working when individual nodes fail, because data is copied elsewhere.
Concurrency	Many transactions on many nodes run at once without corrupting shared data (the concurrency control of Chapter 13, now across machines).
Scalability	Capacity grows by adding nodes.
Network partition	Some nodes can't talk to others because the network between them failed. Don't confuse it with *data* partitioning (15.2).

Scale up vs scale out

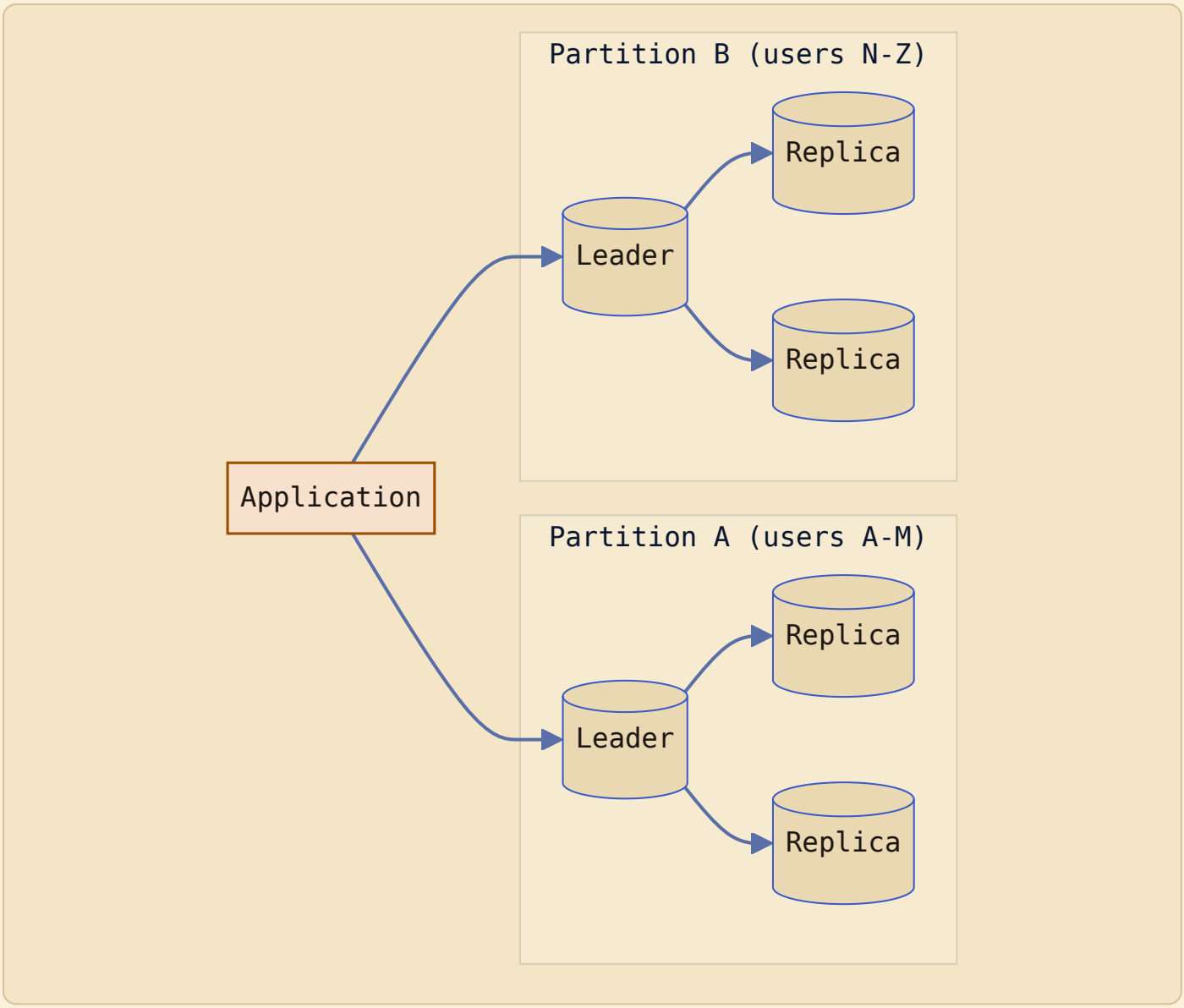
	Vertical scaling (scale up)	Horizontal scaling (scale out)
How	Bigger machine: more CPU, RAM, faster disks	More machines, each holding part of the data/load
Example	Move Postgres from 8 to 64 cores for Black Friday	Add nodes to a Cassandra or CockroachDB cluster as users grow
Limit	The biggest machine you can buy (and its price)	Coordination cost between nodes
Failure	Still one machine = one point of failure	Survives node loss if data is replicated
App changes	None	Often needed (shard keys, cross-node queries)

Distributed databases are built for **scale-out**. Scale-up is simpler and usually comes first — many companies run a large Postgres or MySQL **primary** (the one server that accepts writes) plus **read replicas** (copies that serve only reads) for years before they shard. For a gentle introduction to these system-design building blocks see System Design Fundamentals.

Benefits and challenges

Benefits	Challenges
High availability (the service stays usable almost all the time) — other copies serve if a node or region dies	**Consistency** of copies written in different places (15.5–15.7)
Lower latency (latency = the time one request takes, start to finish) — data near its users (Singapore reads a Singapore node)	**Network latency** — milliseconds per cross-region hop
Load spreading — no node takes every request	**Failure detection** — silent may mean dead or slow (15.8–15.10)
Capacity grows with node count	**Operational complexity** — harder debugging and upgrades

How scale-out works: partition *and* replicate



Partitioning gives each node a *subset* of the data (capacity, parallel work); **replication** stores each subset on *several* nodes (fault tolerance, read capacity). Most real systems do both. *Example:* a food-delivery app in 20 countries partitions menus and orders by country (queries are local, and data-residency laws such as **GDPR** (the EU's General Data Protection Regulation, which restricts where and how personal data is stored) are easier to meet), replicates each partition to three

availability zones (separate data centres in one cloud region, with independent power and networking), and adds nodes to the busiest cities on New Year's Eve.

Self-check

1. *Which two techniques does almost every distributed database combine?* Partitioning (split) and replication (copy).
2. *Why try scale-up first?* No application changes and no distributed-systems problems; its limits are cost and a **single point of failure** (one component whose failure takes everything down).

15.2 Partitioning

What it is

Partitioning splits one large table into smaller pieces (**partitions**) that are stored and managed separately while queries still see one logical table — like a filing cabinet with one drawer per year. Its goals: **faster queries** (a query for March 2023 reads only the 2023 partition — **partition pruning**), **easier maintenance** (archive a year by detaching one partition), **scalability** (partitions can live on different nodes — then it's *sharding*, 15.3), and **fault tolerance** (each partition can be replicated independently).

Horizontal vs vertical

Aspect	Horizontal (split rows)	Vertical (split columns)
Each piece has	All columns, some rows (2023 orders, 2024 orders)	Some columns, all rows; pieces share the primary key (1:1)
Speeds up	Queries that filter on the partition key	Queries that need only a few columns
Cost	Routing; queries without the key hit every piece	Joins to reassemble a full row
Typical use	Huge event/order/log tables, distributed DBs	Wide tables with big text/JSON/blob columns

A vertical split (`code/05_vertical_partitioning.sql`) keeps listing-page columns (`product_id`, `name`, `price`) in `product_core` and the long `description` / `spec_sheet` text in `product_details`, whose primary key also references `product_core`. With 1,000 products, `pg_total_relation_size` shows:

```
 core_size | details_size
-----------+--------------
 128 kB    | 1672 kB
```

The listing page now reads 128 kB instead of about 1.8 MB; a view (`product_full`) joins the halves back when needed. Postgres already moves very large values out of line by itself (**TOAST**, Chapter 12), so a manual split pays off most for medium-sized, rarely read columns.

Partitioning strategies

Strategy	Rule	Good for	Watch out for
Range	Key in an interval, e.g. `2023-01-01 ≤ date < 2024-01-01`	Time data, "last N days", archiving	**Hot partition**: all new writes hit the newest range
List	Key in a set of values, e.g. `region IN ('EU')`	Regions, tenants, data-residency rules	Uneven sizes; new values need a partition (or DEFAULT)
Hash	`hash(key) mod N`	Even spread with no natural range (`order_id`, `user_id`)	Range queries can't be pruned; changing N moves data
Composite	Two levels, e.g. range by month, then hash by server	High-volume time series	More partitions to manage
Round-robin	Row 1 → P1, row 2 → P2, row 3 → P3, row 4 → P1…	Perfectly even bulk loads and parallel scans	Can't find a row by key — every lookup scans all partitions

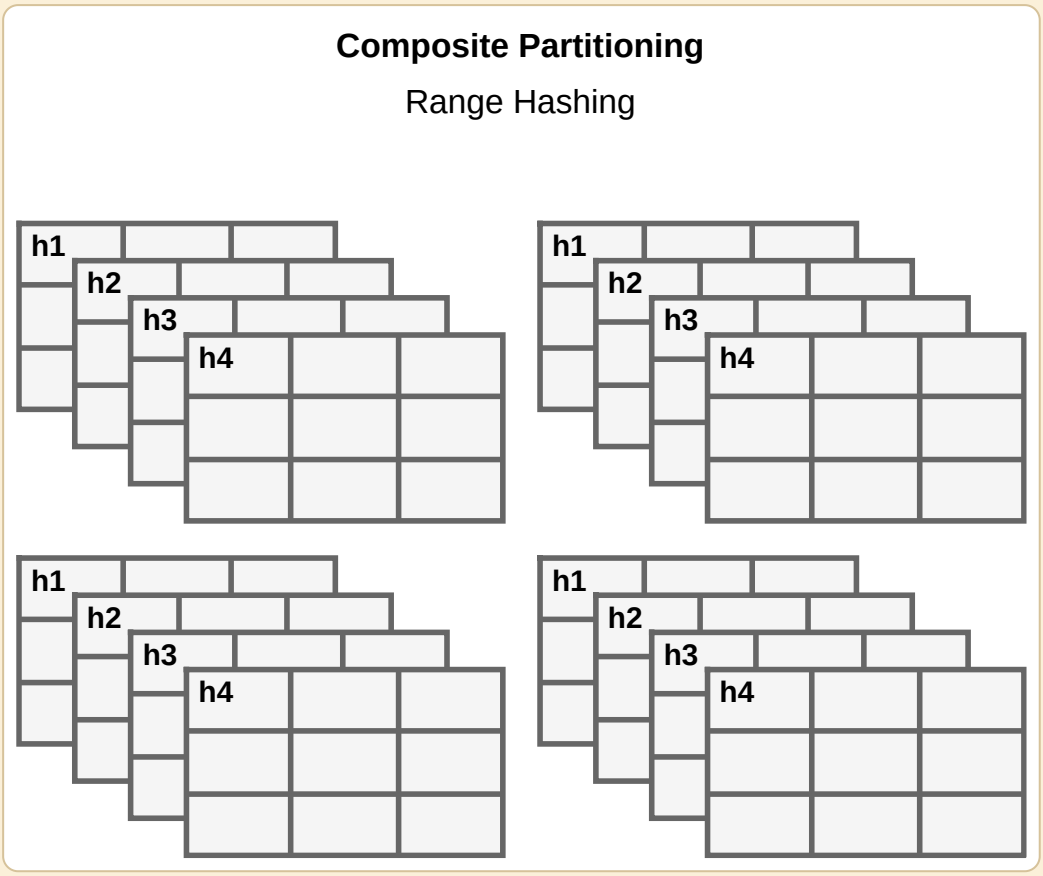


Figure — Composite (range + hash) partitioning: each range partition is itself split into hash buckets h1–h4 (course diagram).

Postgres supports range, list and hash natively, and composite by nesting them; it has no round-robin (that appears in warehouse systems, e.g. Azure Synapse's `ROUND_ROBIN` distribution). **To choose**, look at your most common and most expensive queries: the partition key should be in their `WHERE` clause so pruning works, it should spread data reasonably evenly, and adding/dropping partitions should stay simple.

How it works in PostgreSQL (declarative partitioning)

Declarative partitioning arrived in PostgreSQL 10 (hash partitioning, default partitions and primary keys on partitioned tables in 11). You create a **parent** with `PARTITION BY RANGE | LIST | HASH (columns)` — it stores no rows — and each **partition** with `CREATE TABLE … PARTITION OF parent FOR VALUES …`. Inserts

into the parent are routed to the right partition (a row that fits none is rejected unless a `DEFAULT` partition exists), and a `WHERE` on the partition key lets the **planner** (the part of Postgres that decides how to execute a query) **prune** partitions that can't match.

Range partitioning, with pruning

`code/01_range_partitioning.sql`:

```
CREATE TABLE orders_by_year (
  order_id    int  NOT NULL,
  customer_id int  NOT NULL,
  order_date  date NOT NULL,
  amount      numeric(10,2) NOT NULL,
  PRIMARY KEY (order_id, order_date)          -- a PK must include the partition key
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2022 PARTITION OF orders_by_year FOR VALUES FROM ('2022-01-01') TO ('2023-01-01');
CREATE TABLE orders_2023 PARTITION OF orders_by_year FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');
CREATE TABLE orders_2024 PARTITION OF orders_by_year FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
CREATE TABLE orders_other PARTITION OF orders_by_year DEFAULT;   -- anything outside the ranges

INSERT INTO orders_by_year SELECT * FROM orders;
INSERT INTO orders_by_year VALUES (9001, 1, '2025-03-15', 99);   -- no 2025 partition -> DEFAULT

SELECT tableoid::regclass AS partition, count(*) AS rows,
       min(order_date) AS first_day, max(order_date) AS last_day
FROM orders_by_year GROUP BY 1 ORDER BY 1;
```

```
  partition   | rows | first_day  |  last_day
--------------+------+------------+------------
 orders_2022  | 1016 | 2022-01-01 | 2022-12-31
 orders_2023  |  990 | 2023-01-01 | 2023-12-31
 orders_2024  |  994 | 2024-01-01 | 2024-12-31
 orders_other |    1 | 2025-03-15 | 2025-03-15
```

`FROM … TO …` has an **inclusive** lower and **exclusive** upper bound, so `2023-01-01` belongs to `orders_2023`; `tableoid::regclass` is a hidden column naming the physical table holding each row; and the PK includes `order_date` because each partition enforces uniqueness only on its own rows.

Pruning at plan time, versus a query that doesn't filter on the key:

```
EXPLAIN (COSTS OFF) SELECT sum(amount) FROM orders_by_year
WHERE order_date >= '2023-03-01' AND order_date < '2023-04-01';
EXPLAIN (COSTS OFF) SELECT sum(amount) FROM orders_by_year WHERE customer_id = 7;
```

```
 Aggregate
   ->  Seq Scan on orders_2023 orders_by_year
         Filter: ((order_date >= '2023-03-01'::date) AND (order_date < '2023-04-01'::date))

 Aggregate
   ->  Append
         ->  Seq Scan on orders_2022 orders_by_year_1
               Filter: (customer_id = 7)
         ...   (same for orders_2023, orders_2024 and orders_other)
```

`Append` means "scan each child and concatenate" — all four partitions are read. **Partitioning only helps queries that filter on the partition key.** When the value is known only at run time (a

prepared-statement parameter — a prepared statement is a query parsed and planned once, then run many times with different values plugged into `$1`, `$2` …), Postgres 11+ prunes during execution:

```
SET plan_cache_mode = force_generic_plan;
PREPARE by_day(date) AS SELECT count(*) FROM orders_by_year WHERE order_date = $1;
EXPLAIN (COSTS OFF) EXECUTE by_day('2023-05-05');
```

```
 Aggregate
   ->  Append
         Subplans Removed: 3
         ->  Seq Scan on orders_2023 orders_by_year_1
               Filter: (order_date = $1)
```

Maintenance is where range partitioning shines — archiving a year is a **metadata** change (an edit to the database's own bookkeeping about which tables exist, not to the rows), not a giant `DELETE`:

```
ALTER TABLE orders_by_year DETACH PARTITION orders_2022;         -- instant; now a plain table
DELETE FROM orders_other WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01';
CREATE TABLE orders_2025 PARTITION OF orders_by_year FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
INSERT INTO orders_by_year VALUES (9001, 1, '2025-03-15', 99);
```

Afterwards the parent holds `orders_2023` (990), `orders_2024` (994) and `orders_2025` (1), and the detached `orders_2022` still has its 1,016 rows as an ordinary table. The `DELETE` is needed because Postgres refuses to create `orders_2025` while the DEFAULT partition still holds 2025 rows — better to create partitions ahead of time (a scheduled job or the `pg_partman` **extension** — an add-on package installed into Postgres with `CREATE EXTENSION`). `DETACH PARTITION … CONCURRENTLY` (Postgres 14+) avoids blocking queries on the parent.

List, hash and composite

`code/02_list_partitioning.sql` splits customers by region (`FOR VALUES IN ('NA')`, `('EU')`, `('APAC')`):

```
   partition    |         customers
----------------+---------------------------
 customers_na   | Ben, Elena, Grace, Jack
 customers_eu   | Chloe, Dmitri, Ines, Liam
 customers_apac | Asha, Farid, Hiro, Kavya

 Seq Scan on customers_eu customers_by_region
   Filter: (region = 'EU'::text)

NOTICE:  rejected: no partition of relation "customers_by_region" found for row
```

The last line comes from inserting a `'LATAM'` customer with no matching partition and no `DEFAULT` — a useful safety net that stops data landing in the wrong jurisdiction.

`code/03_hash_partitioning.sql` spreads orders over four partitions (`FOR VALUES WITH (MODULUS 4, REMAINDER 0..3)`): 730 / 752 / 774 / 744 rows — close to even. `WHERE order_id = 2024` prunes to one partition (`Seq Scan on orders_h2`), but `order_id BETWEEN 100 AND 110` produces an `Append` over all four, because hashing scatters neighbouring values.

`code/04_composite_partitioning.sql` partitions server logs by month, and each month is itself declared `PARTITION BY HASH (server_id)` with two leaves (`MODULUS 2, REMAINDER 0/1`):

```
 leaf_partition  | rows | servers
-----------------+------+---------
 logs_2024_01_h0 |  403 | 1,2
 logs_2024_01_h1 |  804 | 3,4,5,6
 logs_2024_02_h0 |  265 | 1,2
 logs_2024_02_h1 |  529 | 3,4,5,6

 Aggregate
   ->  Seq Scan on logs_2024_02_h1 server_logs
         Filter: ((logged_at >= '2024-02-10 00:00:00+00'::timestamp with time zone) AND
                  (logged_at < '2024-02-11 00:00:00+00'::timestamp with time zone) AND (server_id =
3))
```

A filter on both keys prunes to **one leaf of four**. The 2-vs-4 server split also teaches that **hashing evens things out only with many distinct key values** (six servers into two buckets is lumpy). `pg_partition_tree('server_logs')` lists the tree: parent (level 0), months (level 1), leaves (level 2).

Trade-offs and dialects

Postgres limits to know: unique/PK constraints must include every partition-key column; planning time grows with partition count (hundreds are fine, many thousands are heavy); an index on the parent becomes a *local* index on each partition — there is no global index; an `UPDATE` that changes the key moves the row (PG 11+); and `enable_partitionwise_join` / `_aggregate` (off by default) process matching partitions pairwise.

Other dialects *(not run)*: **MySQL** declares `PARTITION BY RANGE/LIST/HASH/KEY (...) (PARTITION p0 VALUES LESS THAN (...), ...)` inside `CREATE TABLE`, and every unique key must include the partitioning columns; **SQL Server** uses a partition function plus a partition scheme (range only); **Oracle** adds interval partitioning (auto-created ranges); **SQLite** has none.

Common mistakes

- A key the hot queries don't filter on (every query scans every partition); partitioning tables under a few million rows at all — a good index is simpler.
- Forgetting next month's partition, so inserts fail or pile into DEFAULT; thousands of tiny partitions; expecting hash partitioning to speed up range scans.

In the interview

30-second answer: "Partitioning splits a big table into physical pieces by a key: range for time, list for categories, hash for even spread, composite to combine. The win is pruning — the planner skips partitions the `WHERE` clause excludes — plus cheap maintenance like dropping a month instantly. It only helps queries that filter on the key, and in Postgres unique keys must include it."

- *Range vs hash for an `events` table?* Range by time if queries are "recent events" and old data is archived; hash by `user_id` if queries are per user and write load must be spread.
- *Hot-partition problem?* Range-by-time sends all current writes to the newest partition; mitigate with time + hash composite.

Self-check

1. *Why is `2023-01-01` in `orders_2023`?* Range upper bounds are exclusive.
2. *What does `Subplans Removed` mean?* Partitions pruned at execution time because the value was a parameter.
3. *Which strategy can't find a row by key without scanning everything?* Round-robin.

TL;DR

Partition by the column your big queries filter on: range for time, list for categories, hash for even spread, composite to combine. Check pruning with `EXPLAIN`; unique keys must include the partition key.

15.3 Sharding

What it is

Sharding is horizontal partitioning where each partition — a **shard** — lives in a **separate database server**, a complete, independent database holding a subset of the rows. A **shard key** decides which shard owns a row, and a **router** sends each query to the right shard(s). Example: `order_id` 1–1,000,000 on shard A and 1,000,001–2,000,000 on shard B; `WHERE order_id = 42` goes only to A.

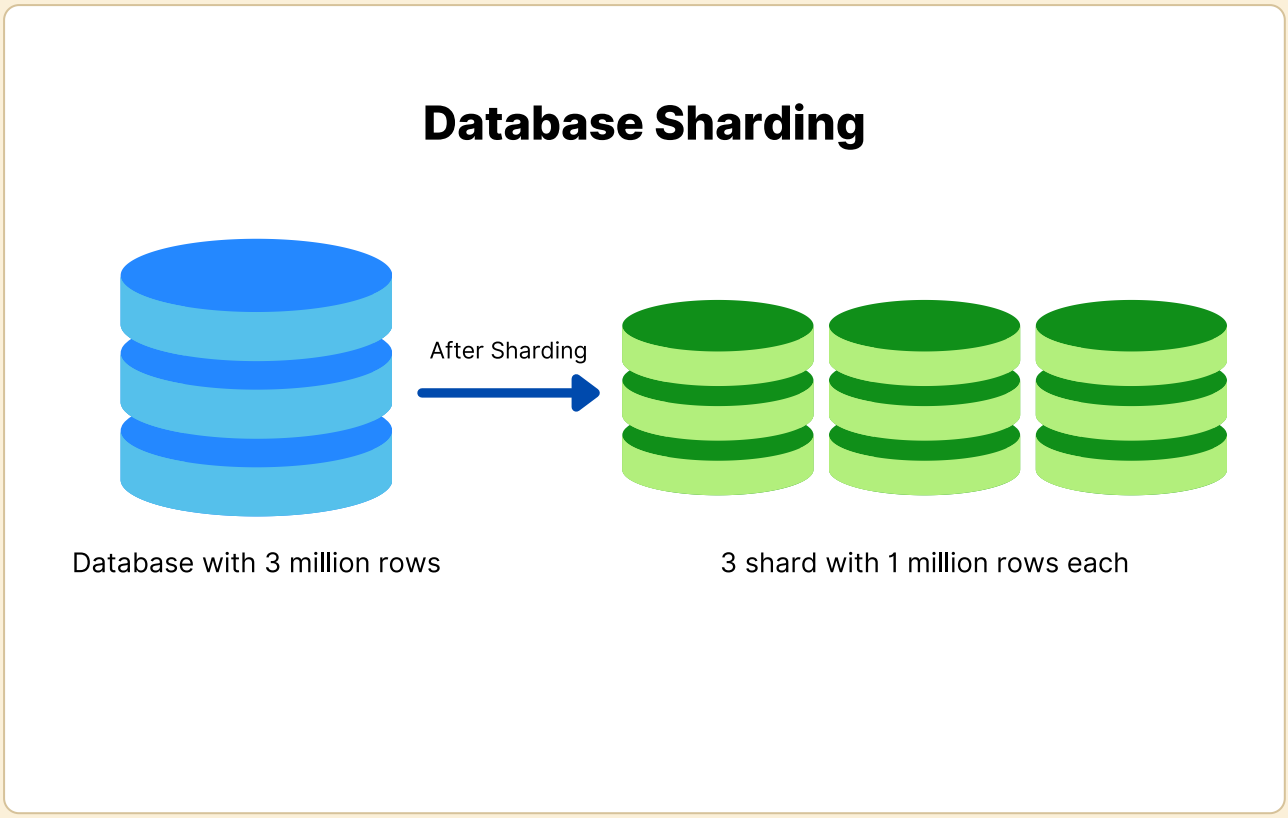


Figure — One database with millions of rows split into three shards, each an independent database (course diagram).

How it works

1. Pick a **shard key** (`user_id`, `tenant_id`).
2. Choose a **mapping** from key to shard: a range table, a hash, or a lookup directory.
3. A **router** applies it: the application, a **proxy** (a middle server that receives client connections and forwards them to the right database — Vitess for MySQL; pgcat/pgdog for Postgres), or the database itself (Citus, MongoDB `mongos`, CockroachDB).
4. Single-shard queries go straight to one shard; queries without the key are **scattered** to all shards and **gathered** — slower.
5. Each shard is **replicated** (15.5) — sharding alone doesn't remove single points of failure; a shard without replicas loses its slice of data when it dies.

Choosing the shard key (the most important decision)

Property	Why	Example
Spreads data and load evenly	Avoids a **hotspot** — one overloaded shard sets the speed of the whole system	`user_id` (millions of values), not `country`
Matches the main query pattern	Most queries hit exactly one shard	B2B SaaS sharded by `tenant_id`
Keeps related data together	Joins and transactions stay on one shard (**co-location**)	A user's orders on the user's shard
Rarely changes / minimises rebalancing	Changing the key or shard count means moving data	Immutable ids, not email addresses

Typical choices: `user_id` for social apps, `tenant_id` for SaaS, time ranges for time-series (but then all current writes hit the newest shard).

Compared with partitioning inside one server, sharding needs a router instead of the planner, turns cross-piece queries into network scatter-gather and cross-piece transactions into distributed commits (2PC, 15.9), and lets one shard fail while the others keep serving (**fault isolation**). Its benefits: **write scaling**, **smaller working sets** (the **working set** is the data a system touches often; here each shard's hot data fits in RAM), fault isolation and more total capacity.

Rebalancing: why `hash % N` hurts and consistent hashing helps

With naive `hash(key) % N`, changing N from 4 to 5 changes the answer for most keys. (A **hash** function turns any key into a fixed-size, random-looking number; the same key always gives the same number.) **Consistent hashing** puts shards and keys on a ring (0 to 2^{32}); each key belongs to the next shard clockwise, so a new shard only takes keys from its neighbour. **Virtual nodes** give each shard many small points on the ring to even out the load (Cassandra, Riak and Dynamo-style systems). `code/consistent_hashing.py` places 100,000 keys, then adds a fifth shard:

The ring is a sorted list of `(hash, shard)` points; `lookup` uses `bisect` (Python's binary search over a sorted list) to find the first point clockwise of `h(key)`:

```
def lookup(self, key):
    i = bisect.bisect(self.hashes, h(key)) % len(self.points)  # next point clockwise
    return self.points[i][1]
```

```
hash % N                    moved  79.7%   max/min shard load 1.01
ring, 1 point per shard     moved   2.8%   max/min shard load 10.44
ring, 200 vnodes per shard  moved  17.1%   max/min shard load 1.28
ideal fraction to move when going 4 -> 5 shards: 1/5 = 20.0%
```

- `hash % N` balances perfectly but moves **80%** of keys — a huge migration.
- One ring point per shard moves little only because the new shard landed on a tiny arc; the biggest shard holds 10× the smallest.
- **200 vnodes** move about 17% (near the ideal 20%) and stay balanced — why real systems use them.

Another approach: a **fixed number of logical partitions** (say 1,024) mapped to servers by a directory; adding a server moves whole partitions and nothing is re-hashed (Citus shards, Vitess keyspace ranges and Kafka partitions work roughly this way). More in DDIA ch.6 — rebalancing.

Sharding Postgres in practice. Core Postgres has no built-in sharding: use application-level routing, the **Citus** extension (`create_distributed_table('orders','customer_id')` co-locates tables sharing the key), partitions whose leaves are `postgres_fdw` foreign tables (15.4), or a Postgres-compatible distributed SQL database (CockroachDB, YugabyteDB — Chapter 16).

Common mistakes

- **Monotonic key + range sharding** (auto-increment id, timestamp): every new row hits the last shard.
- **Low-cardinality keys** (cardinality = the number of distinct values; country, status): too few shards, some huge.
- **Sharding too early**: read replicas, caching (Caching patterns) and a bigger machine are cheaper first steps.
- **Forgetting cross-shard needs**: global unique constraints, cross-shard joins and all-user reports become hard.

In the interview

> **30-second answer:** "Sharding puts horizontal partitions on separate servers so writes and storage scale out. The shard key should spread load evenly, match the main access pattern so most queries hit one shard, and keep related rows together. Queries without the key become scatter-gather and cross-shard transactions need 2PC. For rebalancing I'd use consistent hashing with virtual nodes, or fixed logical partitions mapped to nodes, never `hash mod N`."

- *A celebrity user overloads one shard?* Split that key (`user123#0..9`) to spread writes and merge on read, or cache their reads.
- *Unique ids across shards?* UUIDv7 (a **UUID** is a 128-bit random-looking id that can be generated anywhere without coordination; v7 starts with a timestamp so ids sort by time), Snowflake-style ids (timestamp + machine id + sequence), or per-shard sequences with disjoint ranges.

Self-check

1. *Why is `hash % N` bad for resharding?* Changing N remaps most keys (≈80% going from 4 to 5).
2. *What do virtual nodes fix?* Uneven arcs on the ring, hence uneven load.

15.4 Distributed query processing

What it is

When a query's rows are spread over several nodes, the database must decide **which nodes to ask, what to ask each one, and how to combine the answers** — moving as little data as possible, using nodes in parallel, and still returning a correct answer.

How it works



Step 1 includes **data localisation** (finding which nodes hold each table); step 3 minimises network traffic and time given data location, link speeds and node load; step 5 merges partial results (union, final sort, final sum).

Technique	Idea
Query fragmentation	Independent fragments run simultaneously on different nodes.
Predicate push-down	Apply `WHERE` on the node storing the data, so only matching rows travel.
Aggregate push-down	Each node computes partial `count`/`sum`; only small results travel.
Local (co-located) joins	Both tables sharded on the join key → each node joins its own pieces.
Distributed joins	Otherwise **broadcast** the small table, **repartition (shuffle)** both sides by the join key, or **semi-join**: send only join keys first, then fetch matching rows.
Smart sharding, caching	Keep common queries on one node; reuse frequent intermediate results.

Challenges: data localisation, network latency, a far larger plan space, nodes failing mid-query, and consistency when pieces come from replicas at slightly different moments.

Seeing it in Postgres with `postgres_fdw`

`postgres_fdw` (a **foreign data wrapper**) lets one Postgres query tables on another; the remote tables appear locally as **foreign tables**. `code/06_distributed_query_fdw.sql` points it back at the same private server: schema `shard_b` plays the "remote node", and `remote.orders` is the foreign table:

```
CREATE EXTENSION IF NOT EXISTS postgres_fdw;
CREATE SERVER node_b FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host '/tmp', port '5544', dbname 'ch_15');          -- in real life: another host
-- ... user mapping; shard_b.orders created as a copy of orders ...
IMPORT FOREIGN SCHEMA shard_b LIMIT TO (orders) FROM SERVER node_b INTO remote;

EXPLAIN (VERBOSE, COSTS OFF) SELECT order_id, amount FROM remote.orders WHERE amount > 500;
EXPLAIN (VERBOSE, COSTS OFF) SELECT customer_id, count(*), sum(amount) FROM remote.orders GROUP BY 1;
```

```
 Foreign Scan on remote.orders
   Output: order_id, amount
   Remote SQL: SELECT order_id, amount FROM shard_b.orders WHERE ((amount > 500::numeric))

 Foreign Scan
   Output: customer_id, (count(*)), (sum(amount))
   Relations: Aggregate on (remote.orders)
   Remote SQL: SELECT customer_id, count(*), sum(amount) FROM shard_b.orders GROUP BY 1
```

The `Remote SQL` line shows what was pushed down: the filter (only two columns and matching rows travel) and the whole `GROUP BY` (only 12 rows come back). A **cross-node join** — local `customers` in the USA joined to remote `orders` — returns the right answer (Ben 250 orders / 64,750.00, Grace 250 / 65,000.00, Jack 250 / 64,750.00), but its plan reveals a weakness (a **Hash Join** builds an in-memory lookup table from one side, then probes it with each row of the other):

```
Hash Join
  Hash Cond: (o.customer_id = c.customer_id)
  ->  Foreign Scan on remote.orders o
        Remote SQL: SELECT order_id, customer_id FROM shard_b.orders
  ->  Hash
        ->  Seq Scan on public.customers c
              Filter: (c.country = 'USA'::text)
```

The remote query has **no WHERE**: all 3,000 orders cross the network and only 750 survive. The **semi-join** fix finds the USA customer ids locally first and ships only those:

```
EXPLAIN (VERBOSE, COSTS OFF)
SELECT o.order_id, o.customer_id FROM remote.orders o
WHERE o.customer_id = ANY (ARRAY(SELECT customer_id FROM customers WHERE country = 'USA'));
```

```
 Foreign Scan on remote.orders o
   Output: o.order_id, o.customer_id
   Remote SQL: SELECT order_id, customer_id FROM shard_b.orders WHERE ((customer_id = ANY
($1::integer[])))
   InitPlan 1 (returns $0)
     ->  Seq Scan on public.customers
           Filter: (customers.country = 'USA'::text)
```

The `InitPlan` (a sub-query Postgres runs once before the main query) runs locally; its three ids travel as `$1`, and only the 750 matching orders come back. Distributed **optimisers** (the components that choose the cheapest execution plan — Citus, CockroachDB, Spark, Trino) make such choices automatically from table sizes and data location.

Common mistakes

- Wrapping remote columns in functions the remote side can't evaluate, which blocks push-down — check `Remote SQL` in `EXPLAIN VERBOSE`.
- Joining two large tables sharded on *different* keys in the hot path — co-locate or denormalise.
- `ORDER BY … LIMIT` across shards without per-shard limits — each shard should return its own top N first.

In the interview

30-second answer: "The **coordinator** (the node that receives the query and orchestrates the others) localises the data, splits the query into per-node fragments, pushes filters and partial aggregates to the data, runs fragments in parallel and merges results. Network transfer dominates, so the optimiser prefers co-located joins, broadcasts small tables and uses semi-joins to ship keys instead of rows."

- *Why can't `AVG` be pushed down as `AVG`?* An average of averages is wrong when groups differ in size; push down `SUM` and `COUNT`, divide at the end.

Self-check

1. *Where do you see what `postgres_fdw` pushed down?* The `Remote SQL:` line of `EXPLAIN VERBOSE`.
2. *What does a semi-join ship first?* Only the join keys from the smaller side.

15.5 Replication

What it is

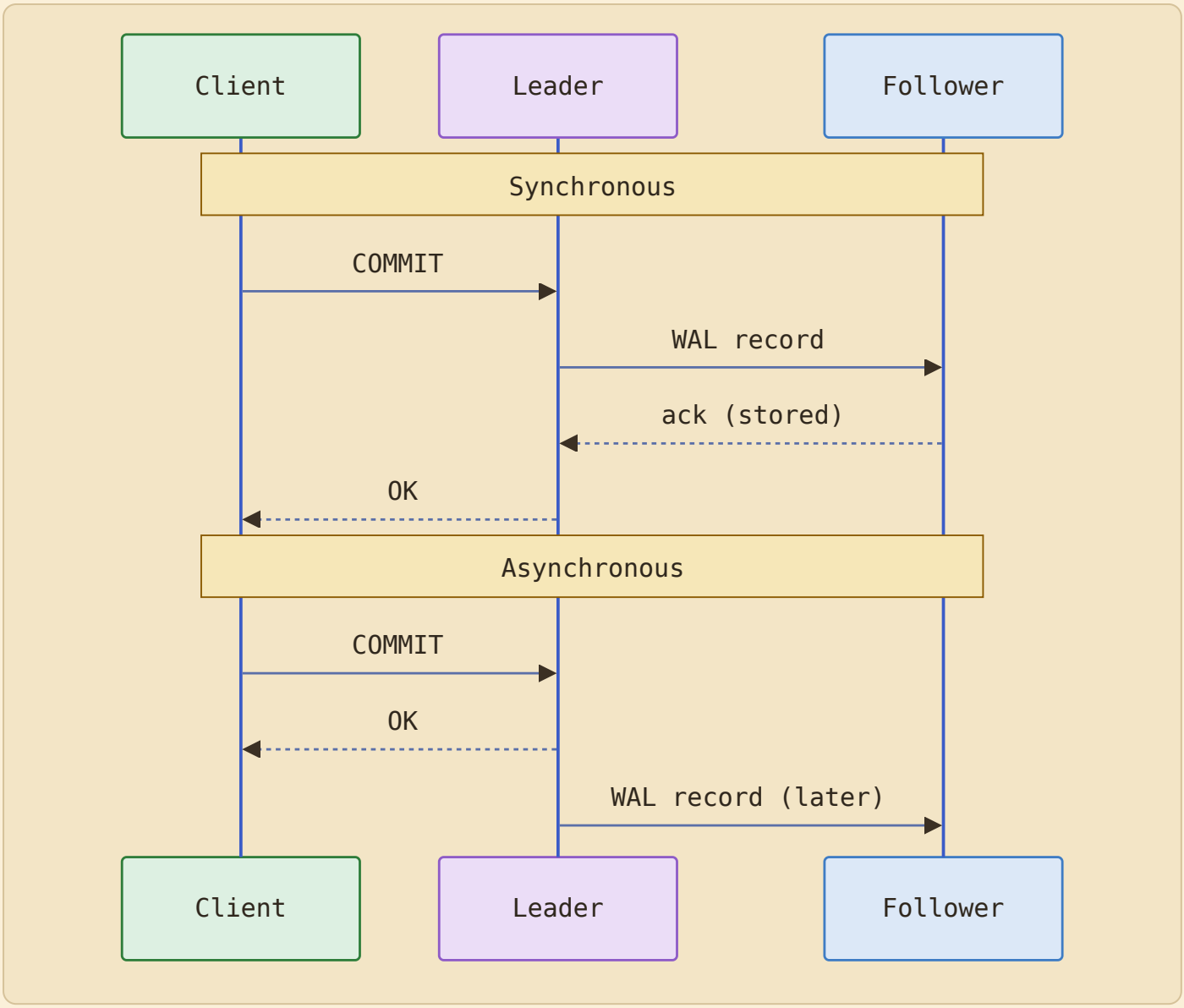
Replication keeps **copies of the same data on several nodes** (**replicas**) — like keeping photos on your laptop, an external drive and the cloud. Benefits: **high availability** (a replica takes over when the primary dies), **read scaling** (reports read from replicas), **lower latency** (a Frankfurt replica serves Europe), **disaster recovery** (**DR** — getting back to work after losing a whole site; here, a copy in another region). Costs: keeping replicas in sync, extra storage and network traffic, and **conflicts** when several copies accept writes.

Kinds of replication (what gets copied, and when)

Type	How it works	Trade-off	Typical use
Full	Every replica holds the whole database	Any replica answers anything / storage × replicas	DR copies
Transactional	Each committed transaction streams to replicas in commit order, near real time	Close and ordered / continuous overhead	OLTP (day-to-day transactional workload) read replicas, Postgres logical replication
Snapshot	Periodically copy a full snapshot	Very simple / only as fresh as the last copy	Nightly reporting copies
Merge	Copies change independently (even offline), then merge by rules	Works disconnected / conflicts	Field and mobile apps
Key-based incremental	Copy rows whose `updated_at`/version is newer than the last sync	Moves little data / **misses hard deletes** (rows physically removed) unless tombstoned (marked deleted instead)	ETL (extract-transform-load pipelines) into warehouses, IoT (internet-connected device) sync

A retailer might mix them: orders and stock transactionally, the catalogue as nightly snapshots, a full replica in another region.

Synchronous vs asynchronous



Synchronous replication returns a commit only when the sync replica(s) also have it: no acknowledged commit is lost, but every write pays a network **round trip** (a message there plus the reply back) and commits **block** if that replica is down (use it for money and orders). **Asynchronous** returns once the leader has it: fast, but the newest commits may be lost if the leader dies, and replicas lag (**replication lag**) — fine for read replicas, cross-region copies and analytics. Most systems are **semi-synchronous**: wait for *one* (or a quorum of) replicas, the rest async. **In PostgreSQL**, physical **streaming replication** ships the **WAL** (write-ahead log — the database's append-only record of every change, 15.9) to **standbys** (replica servers ready to take over), async by default; `synchronous_standby_names` (e.g. `'ANY 1 (s1, s2)'`) makes commits wait, and `synchronous_commit` sets how far: `remote_write` (received), `on` (flushed), `remote_apply` (applied, so standby reads see it). **Logical replication** (`CREATE PUBLICATION` / `SUBSCRIPTION`, PG 10+) streams row changes per table.

Replication topologies

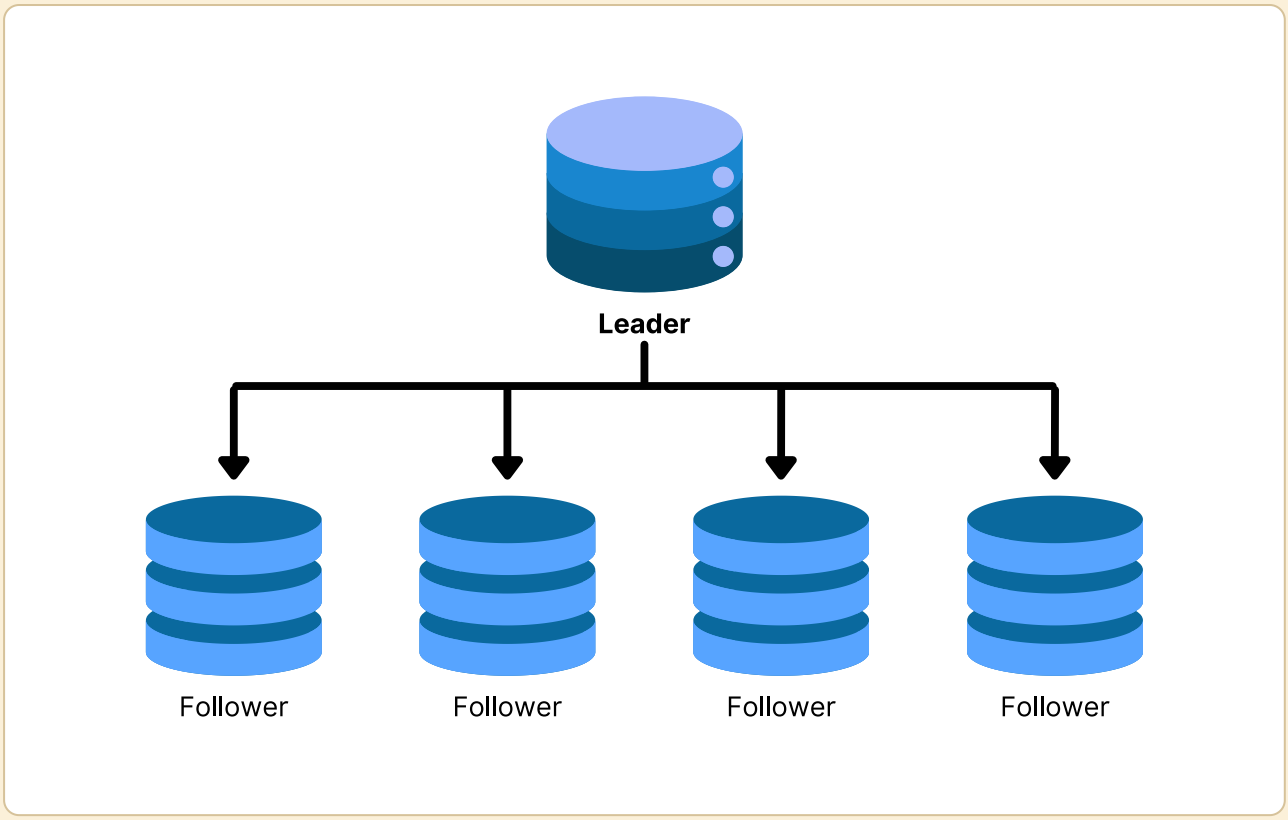


Figure — Single-leader: one leader takes writes and streams them to followers, which serve reads (course diagram).

1. Single-leader (primary/replica; formerly master/slave). All writes go to the leader, which records them in its **replication log / WAL** and ships the log to **followers** (sync or async). Reads may use any node. If the leader fails, **failover** promotes a follower (15.10). Simple, no write conflicts, strong consistency possible; but write throughput is one machine's and async followers can be stale. Examples: PostgreSQL, MySQL, SQL Server Always On, MongoDB replica sets.

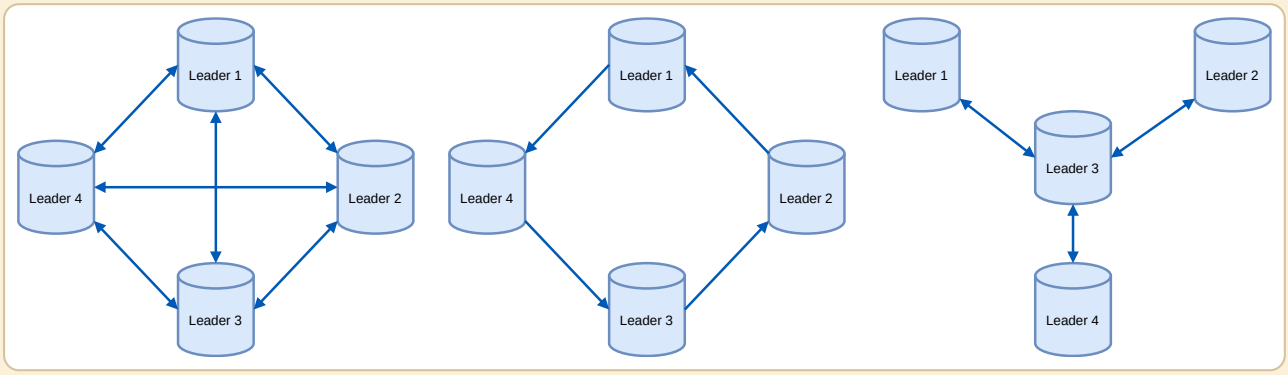


Figure — Multi-leader: several leaders accept writes and replicate to each other (course diagram).

2. Multi-leader (active-active). Several nodes (often one per region) accept writes and replicate to each other. The same row changed at two leaders at once is a **conflict**, resolved by **last-write-wins** (by timestamp — silently drops one write), custom merge code, or **CRDTs** (conflict-free replicated data types, which merge automatically). If one leader fails, clients switch to another. Used for multi-region writes, offline apps (each device is a leader) and collaborative editing. Examples: MySQL Group Replication (multi-primary), Postgres with BDR/pgEdge, CouchDB.

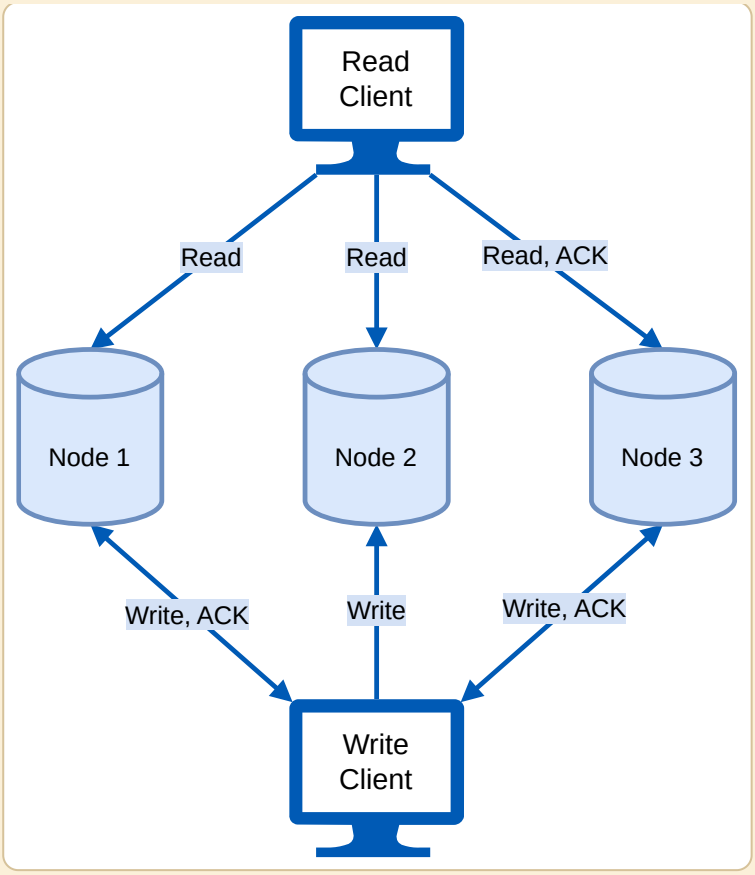


Figure — Leaderless: the client writes to and reads from several nodes directly (course diagram).

3. Leaderless (Dynamo-style). The client (or a coordinator for it) sends each write to all **N** replicas and succeeds once **W** acknowledge; reads ask **R** replicas and keep the newest version. If **R + W > N**, every read set overlaps every write set (the **quorum condition**). Stale replicas are fixed by **read repair** (the reader writes the newest value back), background **anti-entropy** (Merkle-tree comparison — a **Merkle tree** is a tree of hashes where each parent hashes its children, so two replicas can find differing data by comparing a few hashes) and **hinted handoff** (writes for a down node are held and delivered later). Concurrent writes are detected with **version vectors / vector clocks** (a per-node list of counters attached to each value; comparing two lists tells whether one write happened after the other or they were concurrent). Examples: Cassandra, ScyllaDB, Riak, Amazon's original Dynamo (the DynamoDB *service* actually uses a leader per partition). Leaderless systems use **quorums, not consensus** — no Paxos/Raft per write — so they stay available but guarantee less.

`code/quorum.py` simulates N = 3: in each trial a new version reaches only W random replicas, then a read asks R random replicas, keeps the highest version, and counts as stale if that is still the old value:

```
N = 3 replicas, 10,000 trials per setting
 W  R  R+W>N  stale reads
 1  1  False    66.4%
 1  2  False    34.0%
 1  3  True      0.0%
 2  1  False    33.8%
 2  2  True      0.0%
 ...  (every remaining row has R+W>N and 0.0%)

N=3, W=2, R=2: writes and reads still succeed with 1 replica down (2 of 3 answer); with 2 down they
fail -> the system chooses consistency.
```

Every setting with R + W > N had **zero** stale reads. N=3, W=2, R=2 survives one failed replica — Cassandra's `QUORUM` level; `ONE` is fast but possibly stale. Quorums still aren't full **linearizability** (the strongest guarantee, defined in 15.6): sloppy quorums (writes accepted by stand-in nodes when the usual ones are down), concurrent writes and clock-based last-write-wins can produce anomalies (DDIA ch.5).

Choosing a topology: correctness first → single-leader with a sync replica; writes in several regions → multi-leader with a conflict plan; must stay writable through failures and eventual is acceptable → leaderless with quorums.

Replication lag in practice. A user updates their profile, the next page is served by a lagging replica, and the old version appears. Fixes: read your own recent writes from the leader, pin a session to one replica, or wait until the replica's `pg_last_wal_replay_lsn()` reaches the write's LSN — the session guarantees of 15.6.

In the interview

> **30-second answer:** "Replication keeps copies for availability, read scaling and DR. Sync loses no acknowledged commits but slows writes; async is fast but can lose recent commits and serve stale reads, so most systems are semi-sync. Topologies: single-leader (simple, no conflicts), multi-leader (multi-region writes, conflicts), leaderless (quorums with R + W > N, read repair)."

- *A user doesn't see their own comment?* A lagging async replica served the read; use read-your-writes routing.
- *Last-write-wins downside?* Concurrent updates are silently lost, and **clock skew** (machines' clocks disagreeing slightly) can discard the newer one.

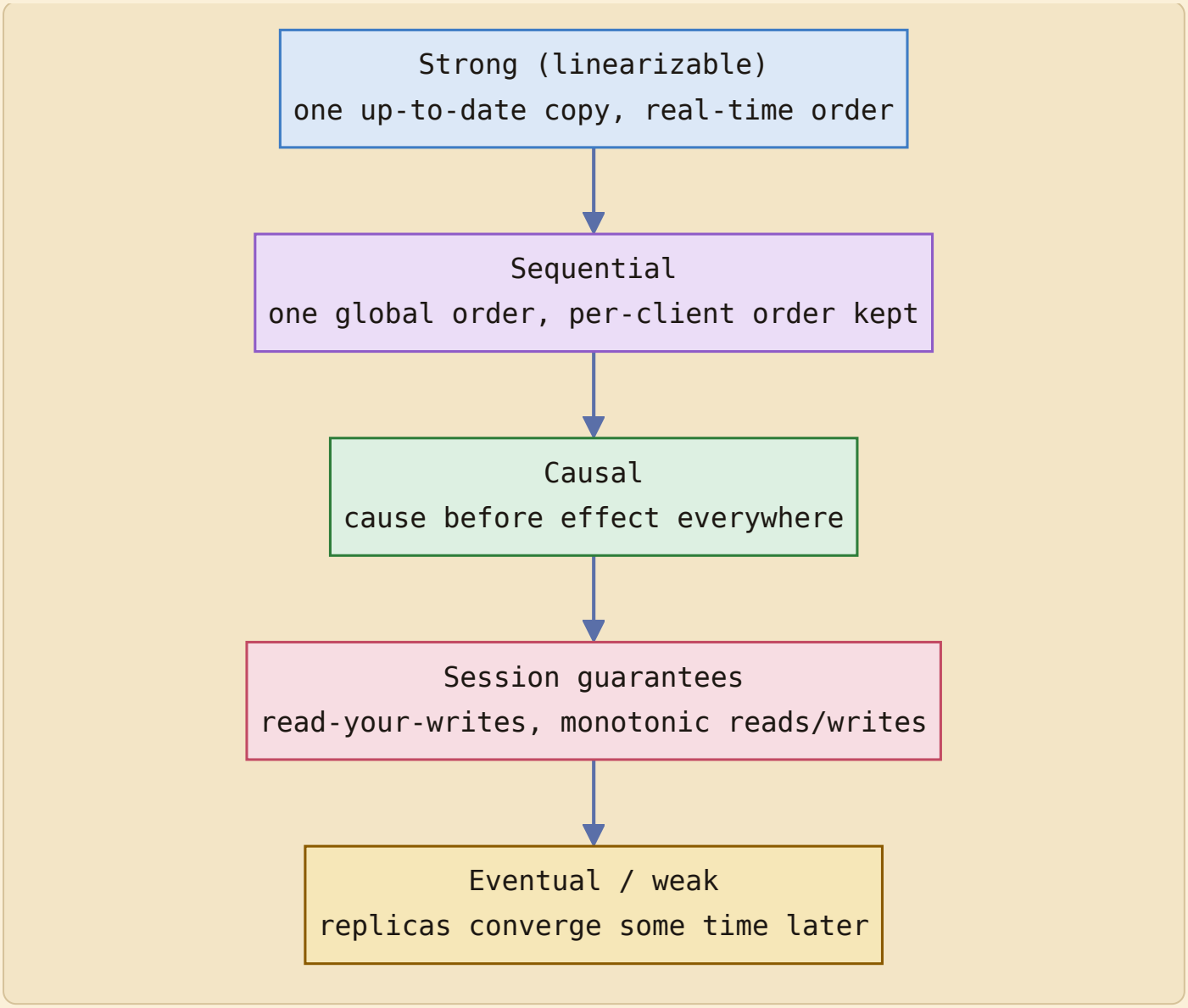
Self-check

1. *Which replication type can miss hard deletes?* Key-based incremental.
2. *N=5: which W and R guarantee overlap?* Any pair with R + W > 5, e.g. W=3, R=3.

15.6 Consistency models

What it is

With several copies, "what will a read return?" has more than one answer. A **consistency model** is the contract: which writes a read is guaranteed to see, and in what order clients observe operations. Stronger models are easier to program against but need more coordination (slower, less available); weaker ones are faster but push work onto the application.


Strong (linearizable)
one up-to-date copy, real-time order
Sequential
one global order, per-client order kept
Causal
cause before effect everywhere
Session guarantees
read-your-writes, monotonic reads/writes
Eventual / weak
replicas converge some time later

The models

Model	Guarantee	How it's built	Pros / cons	Use
Strong (linearizable)	Every read sees the **latest completed write**; the system acts like one copy with operations in **real-time** order (if A finishes before B starts, everyone sees A first)	Synchronous replication or consensus (15.8), atomic operations, one global order	Simplest to reason about, no stale reads / coordination latency, scales less, may be **unavailable** during partitions	Balances, stock at checkout, "is this username taken?", locks. etcd, ZooKeeper writes, Spanner, CockroachDB, single-node Postgres
Sequential	All clients see **the same order**, and each client's operations in its own program order — but not necessarily real-time order	A global sequencer	One agreed history / still needs coordination, no recency promise	CPU memory models, distributed shared memory; rare by name in databases
Causal	If A could have influenced B (same client did A then B, or B read A), **everyone sees A before B**; unrelated (**concurrent**) operations may appear in different orders	Track dependencies with **vector clocks**; delay an update until its dependencies are visible	Little coordination, works across regions, matches intuition / complex tracking, brief divergence	Replies after questions, chat, collaborative editing; MongoDB causally consistent sessions
Weak / eventual	Few promises; **eventual consistency**: if writes stop, replicas converge	Async propagation, anti-entropy, conflict resolution	Lowest latency, highest availability / stale and out-of-order reads, confusing UX (a like count that jumps)	Caches/CDNs (content delivery networks — servers worldwide holding copies of content), counters, analytics, DNS (the internet's name-to-address lookup)

Sequential example: Alice posts P1 then P2, Bob posts Q1; everyone sees one interleaving (say P1, Q1, P2), never P2 before P1 — but a reader just after Q1 finished may not see it yet.

Session (client-centric) guarantees

Weaker than the global models, but they fix the anomalies users notice, **within one session**:

Guarantee	Promise	Anomaly prevented	Example
Read-your-writes	You see your own earlier writes	"My new profile photo reverted"	Profile edits, cart
Monotonic reads	Never an older version after a newer one	A comment *disappears* on refresh	Feeds, notifications, config values
Monotonic writes	Your writes apply in the order issued	"price = 12" applied before the earlier "price = 10"	Version-control commits, debit then credit
Writes-follow-reads	A write after reading X is ordered after X	Reply stored before the post it answers	Comment threads

Session consistency bundles them (Azure Cosmos DB's default level): track the last version/LSN each session saw and route its reads to replicas at least that fresh — coherent for the user and cheap,

though sessions may see different states and routing loses some **load-balancing** (spreading requests across servers) freedom.

How to choose: strong for money and stock; session guarantees where users mostly need their own actions to stick; causal for cause-and-effect content; eventual for high-volume, tolerant data. Many systems let you **choose per operation** — Cassandra consistency levels, DynamoDB `ConsistentRead`, MongoDB read/write concerns, Cosmos DB's five levels.

> **Don't confuse** this "C" with the **C in ACID** (Chapter 13), which means "transactions keep constraints and invariants true". Consistency *models* are about **replicas and ordering**.

In the interview

> **30-second answer:** "A consistency model says what reads can return under replication. Linearizable: one up-to-date copy in real-time order. Sequential: one global order without the real-time promise. Causal: cause before effect. Eventual: convergence only. Session guarantees — read-your-writes, monotonic reads — fix what users see. Stronger costs latency and availability, so I pick per feature."

- *Model for a comment thread?* Causal — replies must never precede their parent.
- *Linearizable vs serializable?* Linearizability is about single objects and real-time recency; serializability is about multi-object transactions behaving as if run one at a time. Both together = "strict serializability" (Spanner).

Self-check

1. *A refresh shows an older comment list. Which guarantee broke?* Monotonic reads.
2. *Sequential vs strong?* Strong also respects real-time order.

15.7 The CAP theorem (and PACELC)

What it is

The **CAP theorem** (Eric Brewer's conjecture, 2000; proved by Gilbert and Lynch, 2002) concerns three properties of a replicated store:

- **C — Consistency** (here *linearizability*; see also System Design Fundamentals for CAP in context): every read sees the most recent successful write. If Priya's balance drops from ₹500 to ₹300, no node may still report ₹500.
- **A — Availability**: every request to a **non-failed** node gets a non-error response, without waiting indefinitely.
- **P — Partition tolerance**: the system keeps working when the network drops or delays messages between groups of nodes.

The theorem: *during a network partition, a system must choose between consistency and availability.*

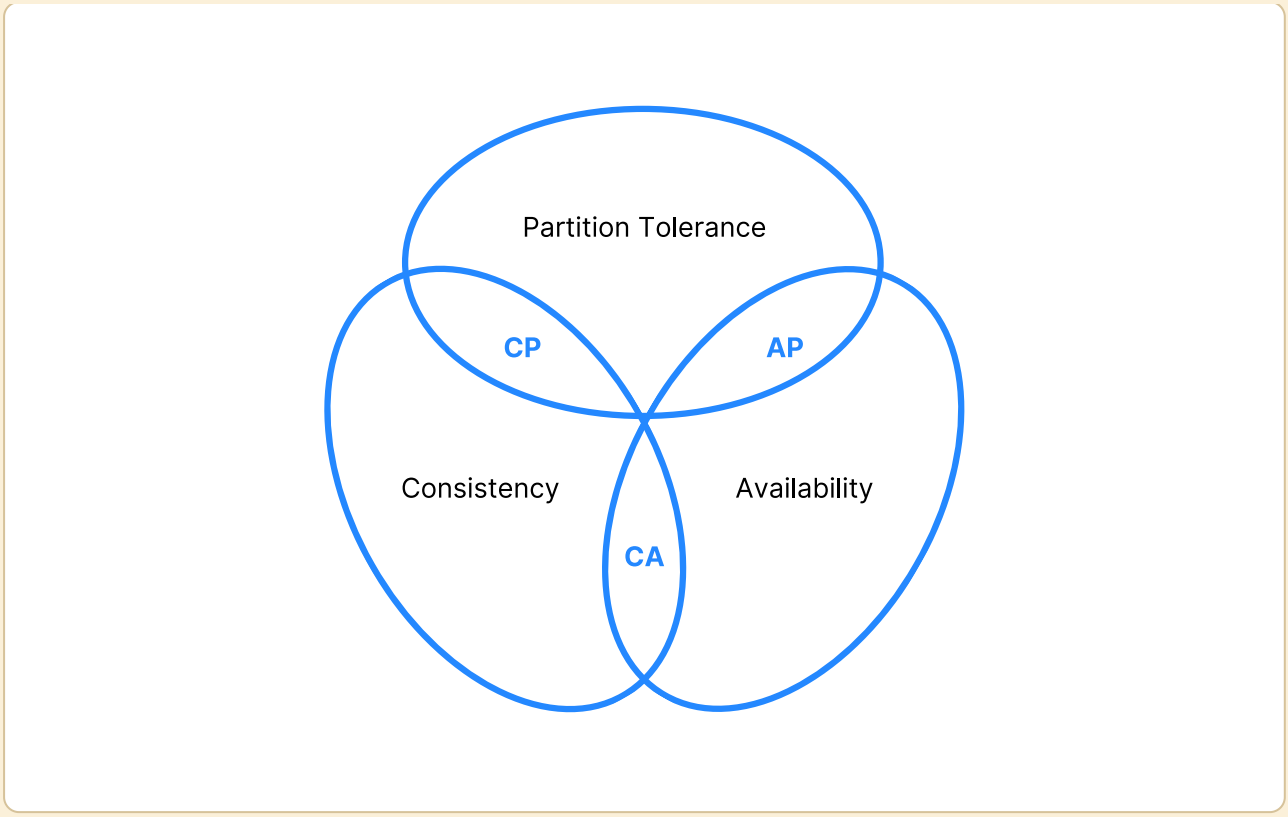


Figure — The three CAP properties and the CP, AP and CA combinations (course diagram).

Why "pick two of three" is misleading

In anything spanning more than one machine, **partitions will happen** (switches fail, cables get cut), so P can't be given up. The real choice is what to do *during* a partition:

Choice	During a partition	Pros	Cons	Examples
CP	Refuse or delay some requests (usually on the minority side) rather than return stale data	Correct, no divergence	Errors/timeouts for some users, coordination latency	etcd, ZooKeeper (small, strongly consistent coordination stores), HBase, Spanner, CockroachDB
AP	Keep answering on both sides, reconcile later	Always responsive	Temporarily inconsistent, needs conflict resolution	Cassandra/ScyllaDB (at `ONE`), Riak, CouchDB, DNS
CA	Only if partitions never happen — effectively one node	Simple and strong	No tolerance of network failure	A single Postgres server (not distributed, so CAP doesn't really apply)

Example: a subscriber counter is kept in two data centres and the link breaks. **AP** lets both sides keep accepting clicks and merges the counts later; **CP** makes the minority side refuse clicks until the link returns.

Nuances: the choice is often **per operation** (CP for "place order and decrement stock", AP for "show reviews"); with **no** partition a system can be both C and A; and CAP's A is strict — many "CP" systems are highly available in practice.

PACELC

Daniel Abadi's **PACELC** (2010): **if Partition, choose A or C; Else, choose Latency or Consistency** — even on a good day, synchronous coordination costs latency.

System	Partition	Else
Cassandra, Riak, DynamoDB (default reads)	PA	EL (fast, possibly stale)
Spanner, CockroachDB, etcd	PC	EC (pay latency for consistency)

In the interview

30-second answer: "CAP says that during a partition a replicated store gives up either linearizable consistency or availability. Partitions are unavoidable, so the real choice is CP or AP — banking and inventory lean CP, feeds and carts lean AP with reconciliation. PACELC adds that without partitions you still trade latency for consistency."

- *Is a single Postgres server CA?* It isn't distributed, so CAP isn't about it; with async replicas plus failover it's neither fully C (can lose recent commits) nor fully A.

Self-check

1. *What does C mean in CAP, and what does PACELC add?* Linearizability; the latency-vs-consistency trade-off in normal operation.

15.8 Consensus and leader election

What consensus is

Consensus gets several nodes to **agree on one value** (or one ordered log) even when some crash or messages are delayed. Everything that needs "exactly one" rests on it — one leader, one commit decision, one log order, one lock owner — so it underpins consistency, fault tolerance, event ordering and coordination. A correct protocol guarantees **agreement** (all non-faulty nodes decide the same value), **validity** (the decision was proposed by someone; if all propose v, it's v), **termination** (non-faulty nodes eventually decide) and **fault tolerance** (it survives a minority of **crash faults**, or fewer than a third of **Byzantine faults** — nodes that lie). It's hard because nodes crash, networks partition and reorder messages, some nodes may be malicious, and every round trip adds latency. The **FLP result** (Fischer, Lynch, Paterson, 1985) proves no **deterministic** (same inputs, always same steps) protocol can *guarantee* termination in a fully asynchronous network if even one node may crash — so real protocols are always *safe* and use timeouts to *usually* make progress.

The algorithms in brief

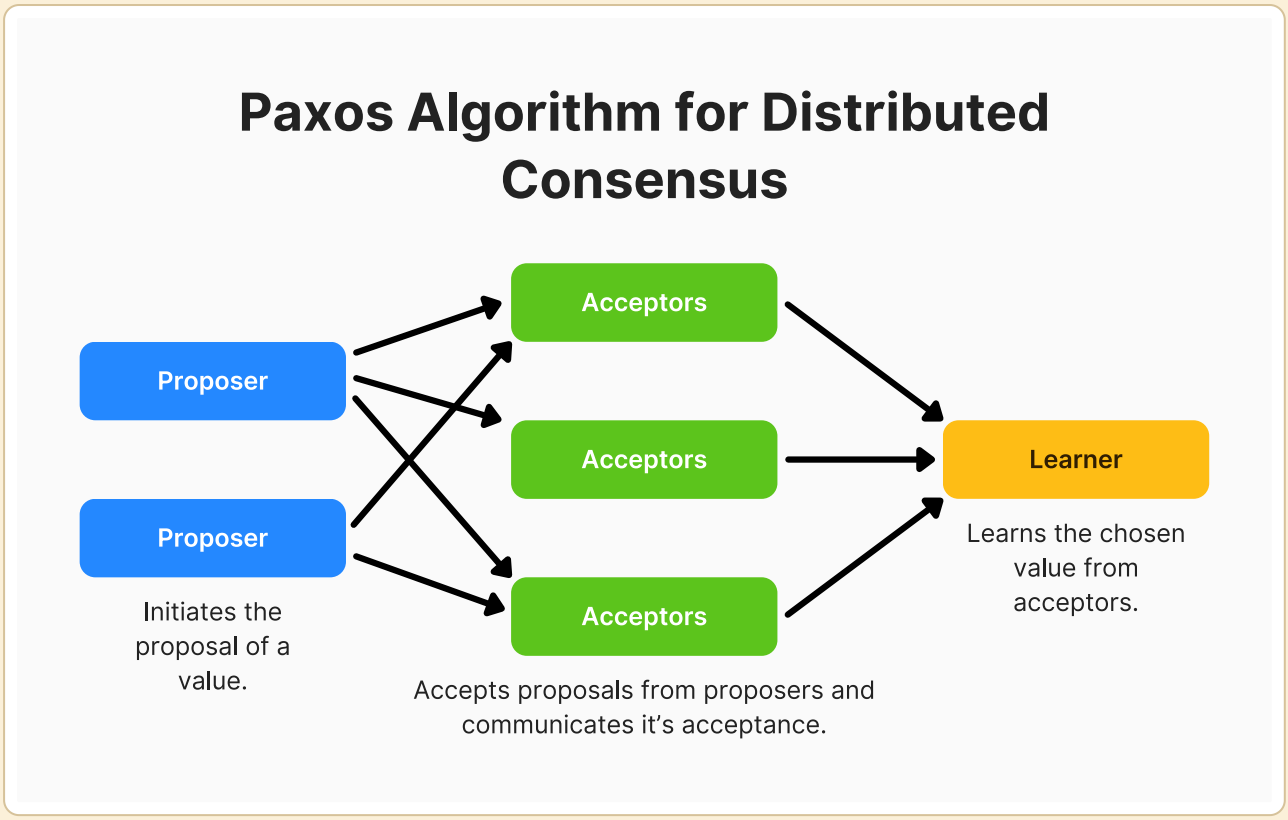


Figure — Paxos roles: proposers send numbered proposals to a set of acceptors, which accept and communicate the chosen value (course diagram).

Paxos (Leslie Lamport; written 1989, published 1998) has **proposers**, **acceptors** and **learners**. (1) **Prepare/Promise:** a proposer sends *prepare(n)* with a unique, higher number to a majority; each acceptor promises to ignore anything below *n* and reports the highest-numbered proposal it already accepted. (2) **Accept:** with a majority of promises the proposer sends *accept(n, v)*, where *v* must be the highest-numbered value reported (its own only if none). (3) Once a **majority accepts**, *v* is chosen and learners are told. It tolerates ⌊(N−1)/2⌋ crashes, needs no synchronised clocks for safety and is formally proven, but it's hard to implement, costs two round trips per decision (**Multi-Paxos** keeps a stable leader to skip phase 1), and duelling proposers can stall it. Used in Google's Chubby and Spanner.

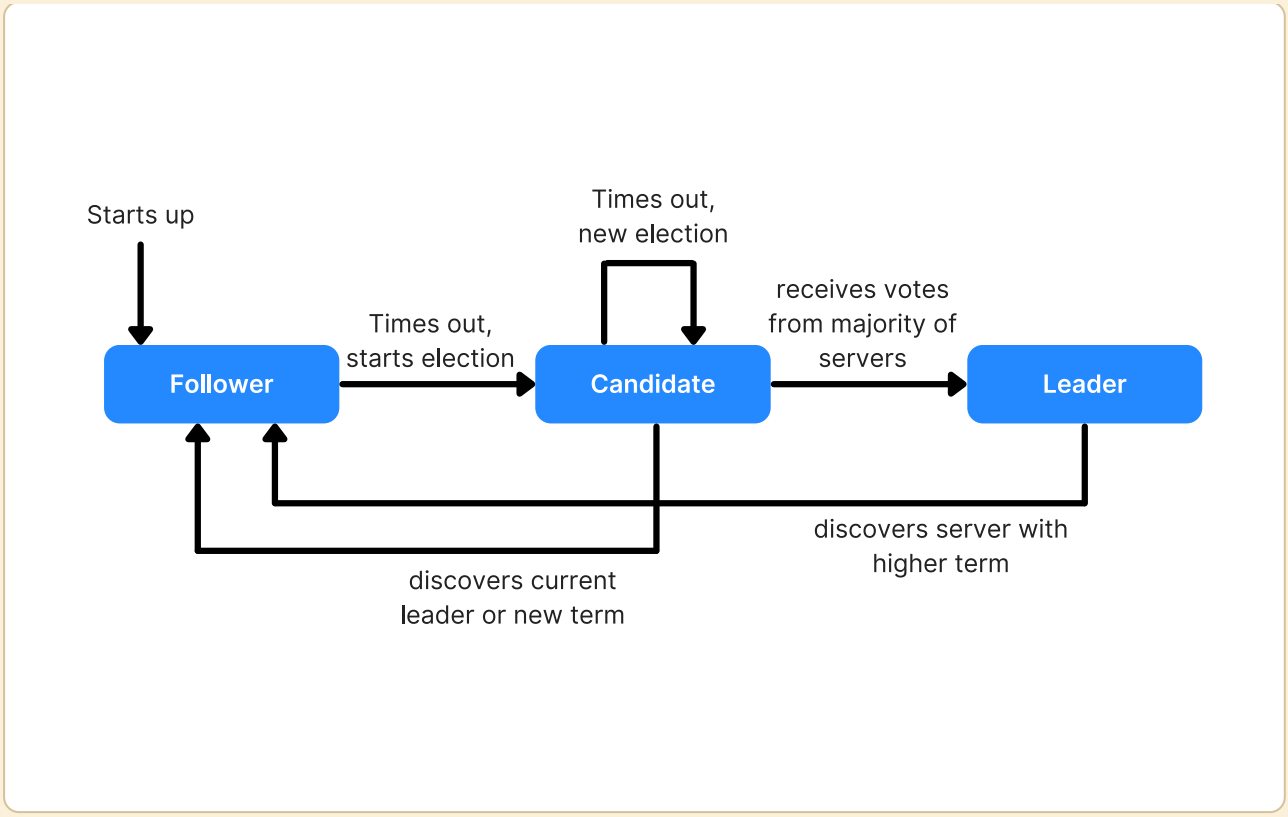


Figure — Raft node states: follower → candidate (on timeout) → leader (on majority vote), and back to follower on discovering a newer term (course diagram).

Raft (Ongaro and Ousterhout, 2014) gives the same guarantees but was designed to be **understandable**. (1) **Election:** time is split into numbered **terms**; a follower that hears no **heartbeat** (a small periodic "I'm alive" message) within its **randomised election timeout** (150–300 ms in the paper) becomes a **candidate**, increments the term and requests votes; each node votes once per term, only for a candidate whose log is at least as up to date as its own; a majority makes a **leader**, and the random timeouts make split votes rare. (2) **Log replication:** the leader sends **AppendEntries** (also the heartbeat); an entry is **committed** once a majority stores it. (3) **Safety:** only a node holding every committed entry can win. Its weaknesses: the leader is a throughput ceiling, unstable networks cause repeated elections, and each commit waits for a majority. Used in etcd (Kubernetes' store), Consul, CockroachDB, TiKV, YugabyteDB and Kafka's KRaft; MongoDB's replica-set protocol is Raft-inspired.

PBFT (Practical Byzantine Fault Tolerance; Castro and Liskov, 1999) tolerates **f lying nodes out of $N \geq 3f + 1$**. In each **view** a **primary** runs *pre-prepare* (assign a sequence number) → *prepare* (replicas broadcast; prepared after 2f matching messages from others) → *commit* (after 2f + 1 matching commits) → *reply* (the client waits for f + 1 identical replies, so one is honest). It costs **$O(N^2)$ messages**, so clusters stay at tens of nodes; used in **permissioned blockchains** (shared ledgers where only approved participants may join; Hyperledger family) and critical control systems.

Proof of Work (Bitcoin) serves **open** networks, where fake identities make vote-counting useless: miners race to find a **nonce** (a throwaway number tried over and over) that puts the block hash under a target, others verify cheaply, and the chain with the most accumulated work wins — no central authority, but heavy energy use, slow finality (a block every ~10 minutes) and exposure to a **51% attack** (whoever controls most of the computing power can rewrite recent history). **Proof of Stake** (Ethereum since the 2022 "Merge") weights votes by locked-up currency instead.

Algorithm	Fault model, tolerance	Cost	Fits
Paxos / Multi-Paxos	Crash, ⌊(N−1)/2⌋	Majority round trips; hard to implement	Databases, lock services
Raft	Crash, ⌊(N−1)/2⌋	Majority round trips, leader-bound	Config stores, distributed SQL
PBFT	Byzantine, ⌊(N−1)/3⌋	$O(N^2)$ messages	Permissioned ledgers
PoW / PoS	Byzantine, open membership (honest majority of hash power / stake)	Slow, energy-hungry / better throughput	Public blockchains

Known members that may crash → Raft or Paxos (what you'll meet inside databases); known members that may lie → PBFT family; unknown members → PoW/PoS. For the full reasoning see DDIA ch.9.

Leader election

A **leader** is the single decider for writes, task assignment or a lock; election picks one and re-picks when it fails. Hard parts: a slow leader looks dead, partitions can create **two leaders** (**split brain**), and clocks differ between nodes.

Strategy	How it works	Strengths	Weaknesses
Bully	The detecting node sends ELECTION to all **higher** ids; if none answers it wins, else the higher node takes over; the winner broadcasts COORDINATOR	Simple, deterministic (highest live id)	Up to **$O(n^2)$** messages; the top node always leads, even if weak
Ring	ELECTION travels round a logical ring, each node substituting its id if higher; whoever gets its **own** id back wins and circulates COORDINATOR	About **2n** messages, equal participation	Slow on big rings; a dead node breaks the ring unless skipped
Randomised	Candidates wait a **random delay**, the first to announce wins, collisions retry	Simple, collisions rare	Non-deterministic (Raft adds majority votes)
ZooKeeper	Each candidate creates an **ephemeral sequential** znode (gone when its session dies); the **lowest** leads; each watches only the znode just **before** its own	Reliable, simple API; ZooKeeper runs the consensus protocol **ZAB**	Extra system to run (same pattern: etcd, Consul, Kubernetes **Lease**)

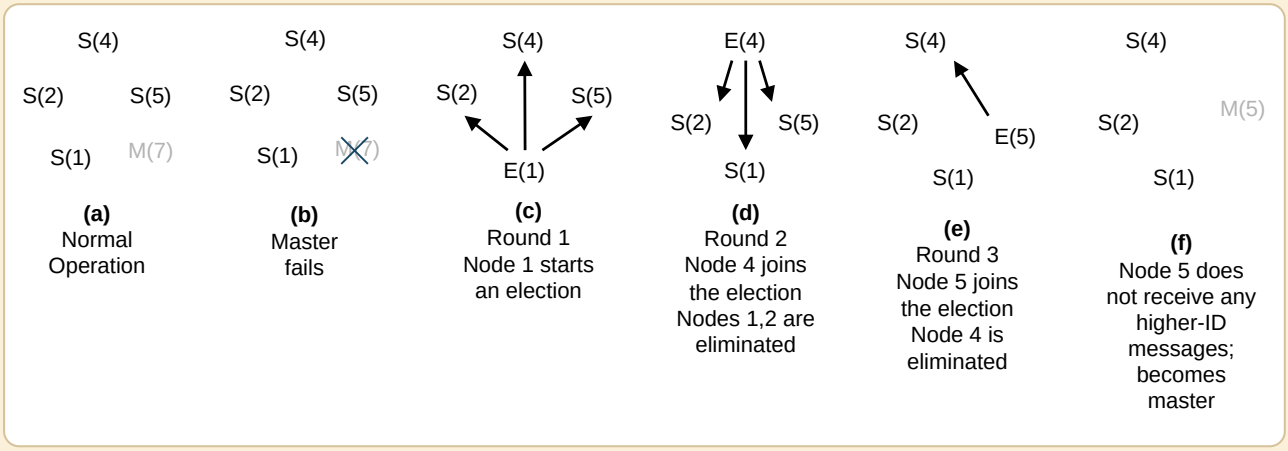


Figure — Bully election: after the leader fails, election messages flow to higher ids until the highest live node announces itself (course diagram).

`code/leader_election.py` counts messages when the highest-id leader has crashed and the *lowest* id notices first (Bully's worst case):

nodes	bully leader	bully msgs	ring leader	ring msgs
5	4	19	4	8
10	9	89	9	18
50	49	2449	49	98

Same leader either way; Bully grows roughly with n^2, Ring with $2n$.

Fencing. A correctly elected leader can be *paused* (long **GC** — garbage-collection pause, when a runtime stops the program to free unused memory — or a **VM** (virtual machine) freeze), replaced, then wake up still believing it leads. Give each new leader a higher term/epoch number (a **fencing token**) and have storage reject writes carrying an older one — Raft terms do exactly this.

In the interview

30-second answer: "Consensus lets nodes agree on a value or log despite failures. Raft elects a leader with randomised timeouts and majority votes and commits an entry once a majority stores it, tolerating $\lfloor (N-1)/2 \rfloor$ crashes; Paxos is the older equivalent. PBFT tolerates liars with $3f+1$ nodes at $O(N^2)$ messages. For leader election I'd use etcd or ZooKeeper and fence stale leaders with epoch numbers."

- *Why 3 or 5 nodes, not 4?* A majority of 4 is 3, so 4 nodes tolerate one failure — no better than 3.
- *What is split brain?* Two nodes both acting as leader after a partition; prevent it with majority quorums and fencing.

Self-check

1. *A 5-node Raft cluster tolerates how many failures?* 2.
2. *PBFT with f = 2 needs at least?* 3f + 1 = 7 nodes.
3. *Which znode does a ZooKeeper candidate watch?* The one just before its own.
4. *What can't be guaranteed in a fully asynchronous system (FLP)?* Termination.

TL;DR

Consensus = majority agreement that survives crashes (Raft/Paxos) or lies (PBFT, 3f+1). Leader election needs consensus underneath and fencing on top — use etcd, ZooKeeper or Consul rather than inventing your own.

15.9 Fault tolerance and recovery

What it is

Fault tolerance is the ability to **keep working correctly while parts are failing**. A **fault** is one component going wrong (dead disk, crashed process, dropped packet); a **failure** is the whole system stopping or answering wrongly — fault tolerance stops the first becoming the second, like a twin-engine plane landing on one engine. Its objectives are **continuity of service**, **data integrity**, **graceful degradation** (recommendations vanish but checkout works) and **recovery**, in support of **availability**, **reliability**, **safety** (failures stay contained — aviation, cars) and **maintainability** (easy repair and upgrade).

Handling a fault has five phases: **detection** (heartbeats, health checks, monitoring) → **diagnosis** (logs, error codes, tools) → **evidence generation** (detailed logs, snapshots, audit trails) → **assessment** (impact analysis, dependency maps) → **recovery** (fail over, restart, reconfigure, restore). Techniques exist at every level: **hardware** (RAID — several disks combined so one can fail without

data loss, see 15.11 — hot-swappable drives, dual power supplies), **software** (exception handling, retries with **backoff** (waiting longer between each retry), timeouts, graceful degradation, **circuit breakers** (stop calling a failing service for a while so it can recover) — see Reliability patterns) and **system** (checkpointing, mirroring, load balancing, replication and failover). The rest of this section covers the database machinery: WAL, checkpoints and distributed commit.

Write-ahead logging (WAL)

The rule: *a change's log record must be safely on disk before the changed data page is written.* The log is an append-only sequential file (fast to write); data pages are written back lazily. After a crash the data files may be stale or half-written, but **the log has the truth**. A transaction changes **pages** (fixed-size blocks, 8 kB in Postgres, in which tables are stored) in memory (the **buffer pool** — the database's in-RAM cache of pages) and creates a log record per change; at **COMMIT** the log up to the commit record is **flushed (fsync'd** — `fsync` is the system call that forces data out of OS caches onto the physical disk), and only then is the commit acknowledged — now **durable** (the D of ACID, Chapter 13). *Example:* Ravi moves ₹5,000 from A to B; "A −5,000", "B +5,000" and a commit record are flushed and he sees "Transfer complete". If the server crashes a millisecond later, before the data pages were written, recovery replays those records.

Redo, undo and LSNs

- **Redo** makes **committed** work survive: recovery re-applies logged changes (**after-images** — the new values; **before-images** are the old values) that may not have reached the data files.
- **Undo** makes **uncommitted** work vanish: before-images restore any in-flight change that reached disk.
- Most databases do both. The classic design, **ARIES** (analysis → redo, "repeat history" → undo the losers), is used by DB2 and SQL Server; InnoDB uses the same redo + undo idea.
- **PostgreSQL's WAL is redo-only.** Thanks to **MVCC** (multi-version concurrency control, Chapter 13 — readers see an older version of a row while a writer creates a new one) an update writes a *new* row version; an aborted transaction is just marked not-committed in `pg_xact`, so its versions are invisible and **VACUUM** (Postgres's background clean-up of dead row versions) removes them later. InnoDB keeps separate **undo logs**, which also serve its MVCC snapshots.

Every WAL record has a **log sequence number (LSN)** — its byte position in the log, always increasing. LSNs **order** redo/undo, tell recovery **what's already applied** (each data page stores the LSN of its last change, so older records are skipped), **mark checkpoints** (replay starts from the checkpoint's redo LSN) and **measure replication lag** (primary LSN vs replica replay LSN).

`code/07_wal_lsn_checkpoint.sql` first prints the server's defaults (`wal_level` = replica, `synchronous_commit` = on — commit waits for the WAL flush, `checkpoint_timeout` = 5min, `max_wal_size` = 1GB, `full_page_writes` = on), then measures WAL growth:

```
INSERT INTO lsn_marks VALUES ('before', pg_current_wal_insert_lsn());
INSERT INTO ledger SELECT i, 'acct-' || (i % 5), (i % 7) - 3 FROM generate_series(1, 5000) i;
INSERT INTO lsn_marks VALUES ('after 5,000 inserts', pg_current_wal_insert_lsn());

SELECT a.label AS from_mark, b.label AS to_mark,
       pg_wal_lsn_diff(b.lsn, a.lsn) > 0         AS wal_grew,
       pg_wal_lsn_diff(b.lsn, a.lsn) > 5000 * 50 AS more_than_50_bytes_per_row
FROM lsn_marks a, lsn_marks b WHERE a.label = 'before' AND b.label = 'after 5,000 inserts';

CHECKPOINT;
SELECT redo_lsn >= (SELECT lsn FROM lsn_marks WHERE label = 'after 5,000 inserts')
       AS redo_point_moved_past_our_inserts
FROM pg_control_checkpoint();
```

```
 from_mark |       to_mark       | wal_grew | more_than_50_bytes_per_row
-----------+---------------------+----------+---------------------------
 before    | after 5,000 inserts | t        | t

 redo_point_moved_past_our_inserts
-----------------------------------
 t
```

`pg_current_wal_insert_lsn()` returns the current end of WAL (like `0/1A2B3C4`; booleans are printed because exact LSNs differ per run). Each row cost over 50 bytes of WAL — row data, index entries and headers. With `full_page_writes = on`, the first change to a page after a checkpoint logs the **whole page**, guarding against **torn pages** (pages only partly written when power was lost, mixing old and new bytes). After `CHECKPOINT` the **redo LSN** is past our inserts, so a crash now would start replay there.

Checkpointing

A **checkpoint** flushes all **dirty pages** (pages changed in memory but not yet written to disk) and logs *"everything before LSN X is on disk"*, so recovery replays only from there and older WAL (not needed by replicas or archiving) can be recycled.

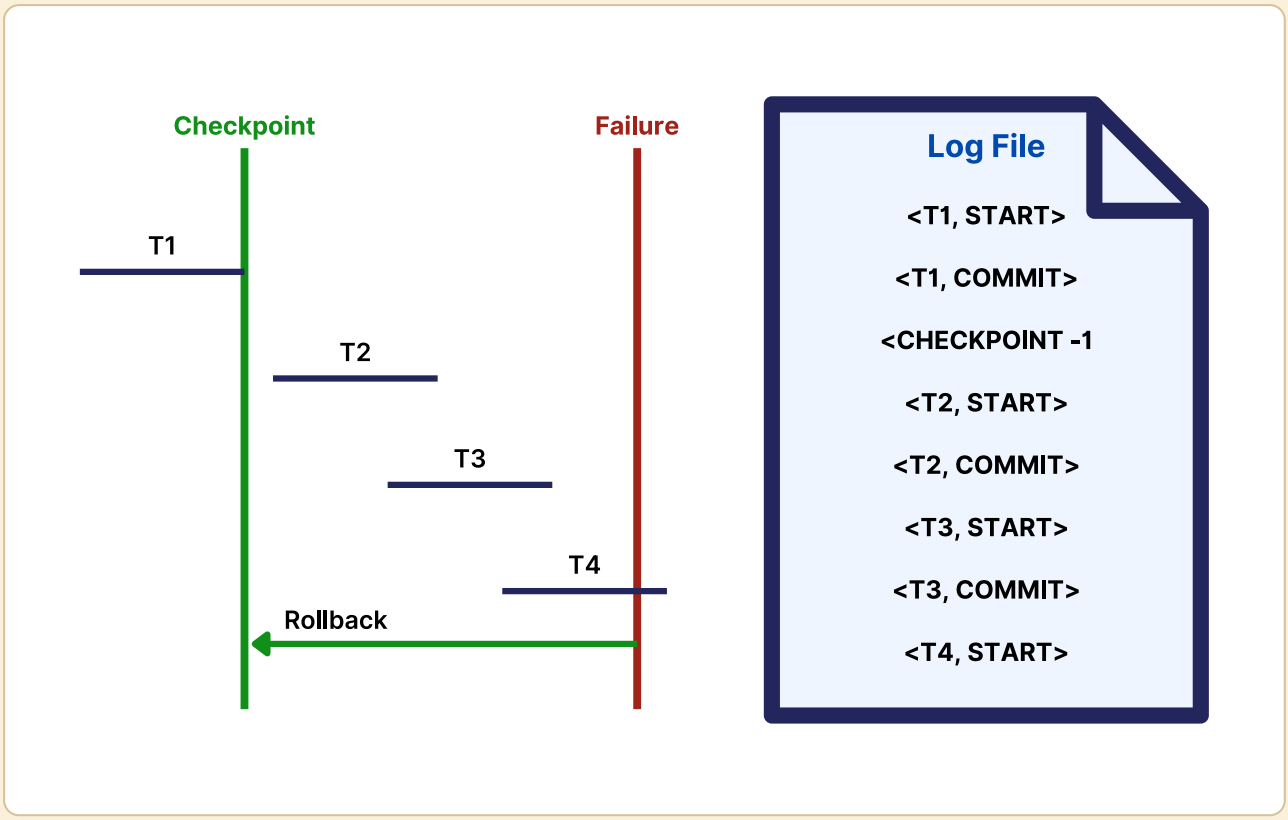


Figure — T1 finishes before the checkpoint, T2 and T3 commit after it, T4 is still running at the failure and is rolled back (course diagram).

At the failure: **T1** is already in the data files (ignored); **T2 and T3** committed after the checkpoint and may exist only in the log (**redone**); **T4** never committed (**undone** — in Postgres, simply invisible). `code/wal_recovery.py` runs this on a toy log — T1 (A→B 300) commits before the checkpoint, T2 (C→D 50) after it, and T3 (A→C 20) is interrupted after one of its pages was flushed — with a forward **redo** pass over updates after the checkpoint and a backward **undo** pass using before-images of transactions that never committed:

```
on disk after crash : {'A': 580, 'B': 400, 'C': 500, 'D': 0}
redo  LSN 206 T2: C = 450
redo  LSN 207 T2: D = 50
redo  LSN 210 T3: A = 580
undo  LSN 210 T3: A = 600
recovered state     : {'A': 600, 'B': 400, 'C': 450, 'D': 50}
money conserved     : True
```

T1 needed nothing, T2 was replayed, T3's early-flushed change was rolled back, and the 1,500 total is intact.

Checkpoints come in two types. A **sharp (consistent)** checkpoint pauses new work, flushes every dirty page and resumes — simple and fully consistent, but a visible stall. A **fuzzy** checkpoint logs a start point, flushes pages *while work continues*, then logs the end; recovery starts from the start point — no downtime, more bookkeeping. PostgreSQL uses the fuzzy style: every `checkpoint_timeout` (5 min default) or when `max_wal_size` of WAL accumulates, spreading writes over `checkpoint_completion_target` (0.9 of the interval by default since PG 14).

Distributed checkpointing. The nodes' saved states must form a **consistent global state**: if node 1's checkpoint records *sending* a message, node 2's must record *receiving* it. **Coordinated** checkpointing (a coordinator, or markers along channels as in the Chandy–Lamport snapshot) is always consistent at the cost of a short pause; **uncoordinated** checkpointing is cheaper but recovery must search for a consistent combination and can cascade rollbacks (the **domino effect**). *Example:* payment, stock and email nodes restore from one global checkpoint, so you never get "stock decremented but payment never recorded" (Apache Flink — a stream-processing engine — uses barrier checkpoints that work this way).

Coordinated commit: two-phase commit (2PC)

Distributed recovery is hard because networks partition, nodes fail and recover at different times, many transactions run concurrently, and every node must agree to commit or abort. When one transaction changes data on **several nodes** (a transfer from an account on node 1 to one on node 2), atomicity must span machines. **2PC** uses a **coordinator**:

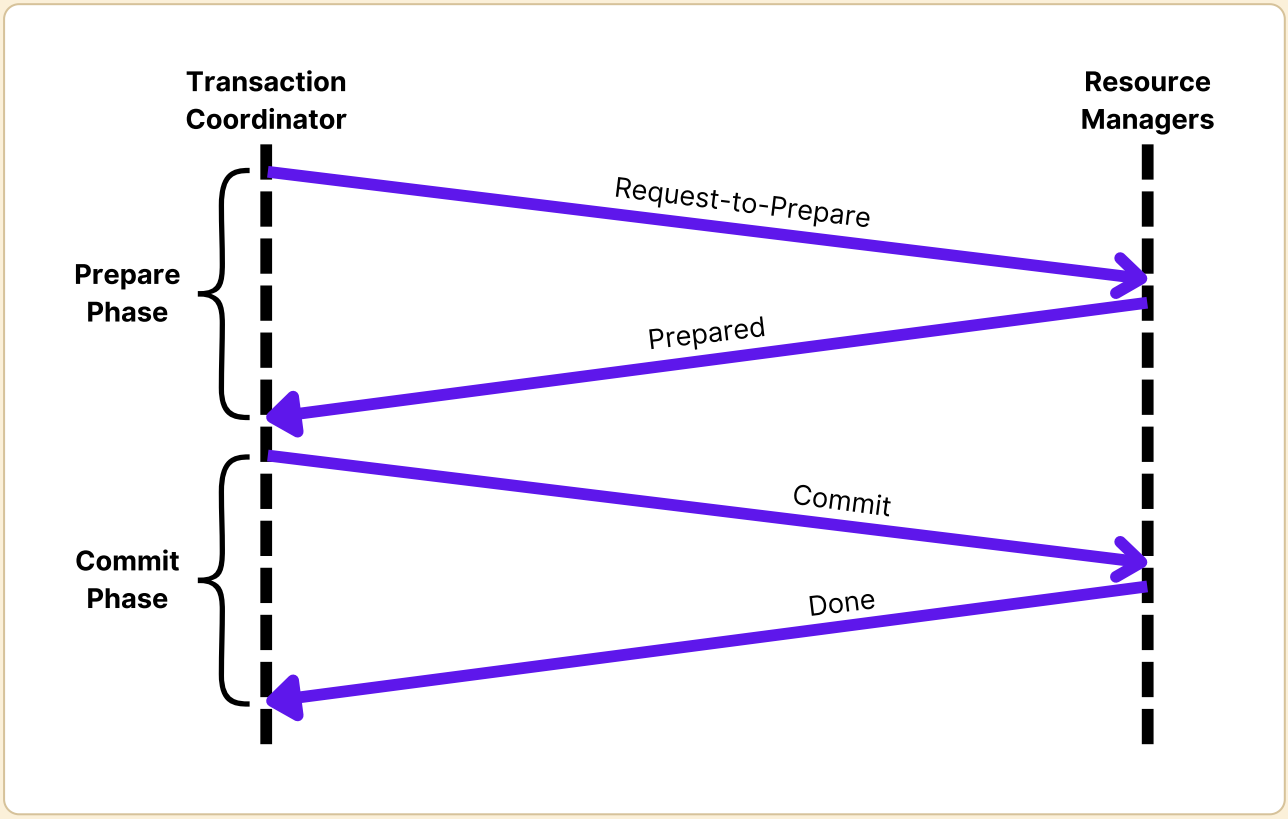


Figure — 2PC message flow: phase 1 asks every participant to prepare and collects their votes; phase 2 sends the commit and waits for acknowledgements (course diagram).

1. **Phase 1 — prepare (vote).** "Can you commit?" Each participant makes its changes durable, logs **PREPARED** and votes **YES** — from then on it may not decide alone — or votes **NO** (constraint failure, disk full).
2. **Phase 2 — decide.** All YES → the coordinator logs **COMMIT** and tells everyone to commit; any NO or timeout → it logs **ABORT** and everyone rolls back.

Recovery: a participant restarting without a PREPARED record aborts alone; one *with* PREPARED must **ask the coordinator**. If the coordinator crashes, participants that haven't voted can abort, but those that **voted YES must wait** — guessing could break atomicity. That wait, with **locks** held (locks stop other transactions touching the same rows), is **2PC's blocking problem**; network delays and partitions also stall decisions. `code/two_phase_commit.py` shows all three cases:

```
--- happy path: debit on node 1, credit on node 2
  ...
  phase 2 final states: {'node1': 'committed', 'node2': 'committed'}
--- one participant votes no (e.g. constraint check failed)
  ...
  phase 2 final states: {'node1': 'aborted', 'node2': 'aborted'}
--- coordinator failure after the votes
  phase 1 votes: {'node1': True, 'node2': True}
  coordinator crashes before sending a decision
  node1: prepared, holding locks, cannot commit or abort alone -> BLOCKED
  node2: prepared, holding locks, cannot commit or abort alone -> BLOCKED
```

2PC in PostgreSQL — the participant side, driven by an external transaction manager *(not run: it needs `max_prepared_transactions > 0`, and the book server keeps the default 0)*:

```
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE id = 'A';
PREPARE TRANSACTION 'transfer-42';   -- phase 1: durable, survives a crash, locks kept
COMMIT PREPARED 'transfer-42';       -- later, on the coordinator's decision (or ROLLBACK PREPARED)
```

Java XA/JTA transaction managers (XA is the standard interface for distributed transactions; JTA is Java's API for it) and MySQL's `XA START … XA PREPARE … XA COMMIT` work the same way. Forgotten prepared transactions hold locks and block VACUUM — monitor `pg_prepared_xacts`.

Three-phase commit (3PC)

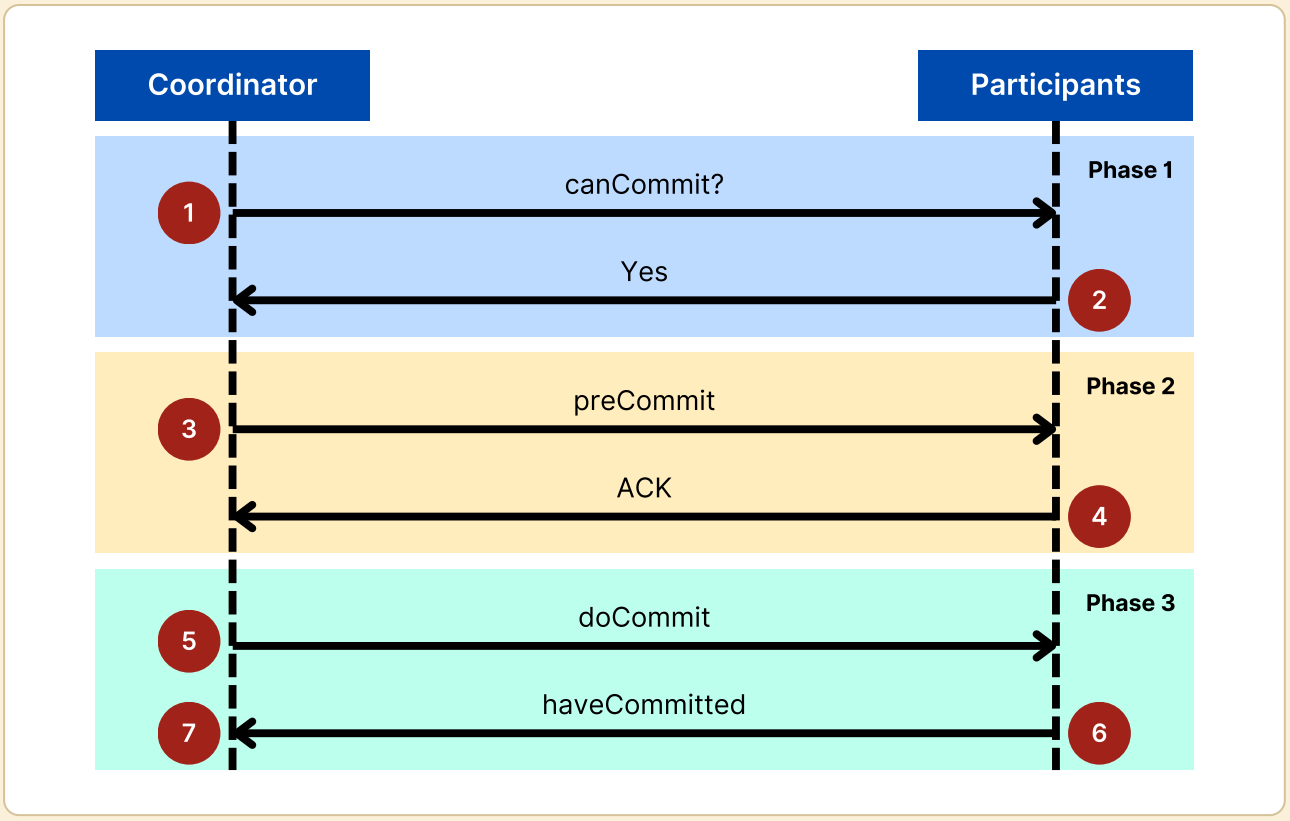


Figure — 3PC: canCommit?/Yes, then preCommit/ACK, then doCommit/haveCommitted (course diagram).

3PC inserts a **PreCommit** step between voting and committing: (1) **CanCommit** — vote as in 2PC; (2) **PreCommit** — if all voted YES, participants acknowledge and log that the outcome will be commit; (3) **DoCommit** — the final commit. If the coordinator dies, participants that logged pre-commit know everyone voted YES and can commit after a timeout; if no live participant has seen pre-commit, nobody can have committed, so they abort. This is **non-blocking** — *provided* message delays are bounded and nodes only crash. Under a real **partition** both sides can time out and decide differently, so 3PC is rarely used. Modern systems make the coordinator fault-tolerant by **running the commit decision through consensus** (Spanner: 2PC over Paxos groups; CockroachDB: a transaction record replicated by Raft), and cross-service workflows use **sagas** (compensating steps) — see 2PC and Saga patterns.

In the interview

30-second answer: "Databases survive crashes with write-ahead logging: the log is flushed before commit is acknowledged and data pages are written later. Recovery starts at the last checkpoint, redoes committed changes and undoes uncommitted ones — Postgres needs no undo thanks to MVCC. Across nodes, 2PC gives atomic commit, but prepared participants block if the coordinator dies, so real systems replicate the decision with Paxos/Raft or use sagas."

- *What does* `synchronous_commit = off` *trade?* Commits return before the WAL flush: a crash can lose the last fraction of a second of commits, never corrupt data.

Self-check

1. *T2 committed after the last checkpoint; T4 hadn't committed. At a crash?* T2 is redone, T4 undone (invisible in Postgres).
2. *A 2PC participant voted YES and lost the coordinator. Can it abort?* No — it must wait.
3. *Sharp vs fuzzy checkpoint?* Sharp pauses work to flush; fuzzy flushes while work continues.

TL;DR

Log first, write data later; checkpoints bound recovery; recovery = redo committed + undo uncommitted. 2PC blocks on coordinator failure and 3PC isn't partition-safe, so modern systems put the commit decision on a consensus group.

15.10 High availability: replication plus failover

What it is

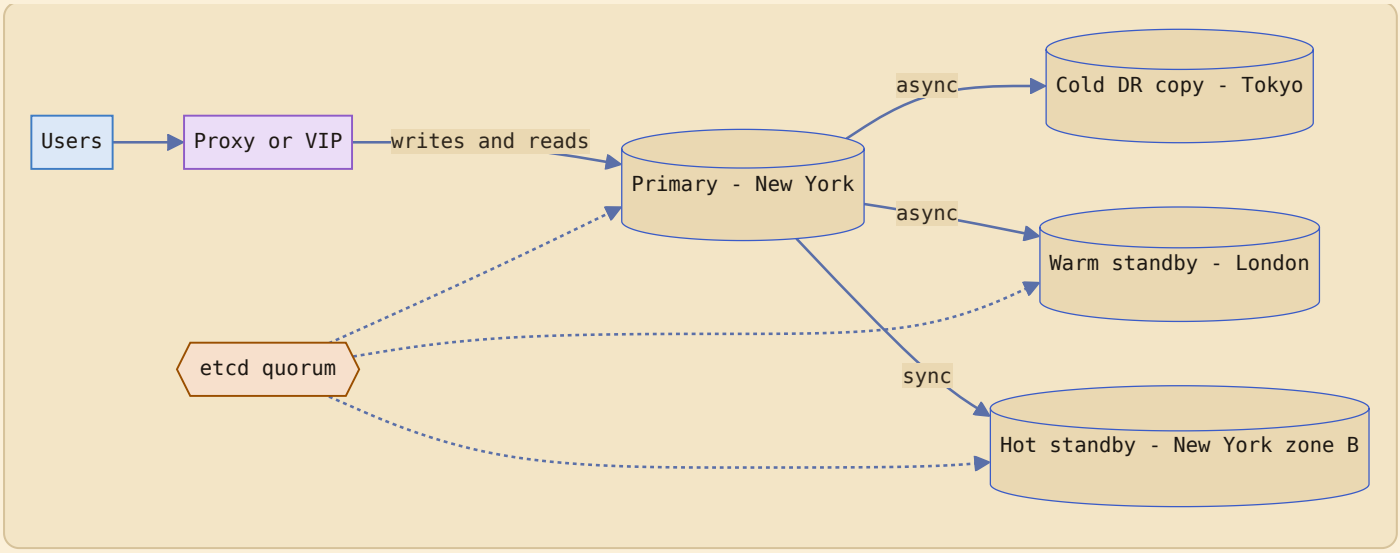
High availability (HA) keeps the service up despite failures, measured in "nines" (the percentage of time the service is up): 99.9% ≈ 8.8 hours of downtime a year, 99.99% ≈ 53 minutes. Two targets make it concrete: **RPO** (recovery point objective — how much *data* you may lose; synchronous replication gives RPO = 0) and **RTO** (recovery time objective — how long you may be *down*; hot standbys and automatic failover make it short). HA = **replication** (15.5; a copy exists) + **failover** (switch to it). A sync standby has every acknowledged commit at failover (RPO 0, slower commits — typical for a local standby in financial systems); an async one may miss the last commits (RPO > 0, fast commits — remote regions, read replicas).

Failover

Type	Standby state	Recovery time	Cost
Cold	Not running; must start and restore/catch up	Minutes to hours	Cheapest
Warm	Running and receiving updates, but needs some catch-up or config	Seconds to minutes	Medium
Hot	Running, in sync, ready to serve immediately	Seconds	Highest

1. **Heartbeats** — several missed "I'm alive" signals raise suspicion.
2. **Decision** — a **quorum** (majority of nodes or witnesses) must agree the primary is dead, preventing **split brain**; **priority rules** pick the standby (highest replayed LSN, preferred zone).
3. **Promotion** — Postgres: `pg_ctl promote` or `SELECT pg_promote()` (PG 12+).
4. **Fencing** the old primary — STONITH ("shoot the other node in the head"), lease revocation, or blocking at the proxy.
5. **Redirecting clients** — DNS, a **virtual IP** (an address that can be moved from one machine to another), a proxy (HAProxy, PgBouncer) or **service discovery** (a registry clients ask "where is the primary right now?").

Core PostgreSQL has **no automatic failover** (switching to a standby without a human — see the steps above); use **Patroni** (leader state in etcd/Consul/ZooKeeper — consensus-backed), **repmgr** or **pg_auto_failover**, or a managed service (Amazon RDS/Aurora, Cloud SQL, Azure Database). **Best practices:** redundancy across failure domains (racks, zones, regions), load-balanced reads with writes routed to the current primary, monitoring (replication lag, heartbeats, disk), **backups too** (a bad `DELETE` replicates in milliseconds — HA is not a backup), and **regular failover tests** (game days, **chaos experiments** — deliberately breaking things in a controlled way to prove recovery works).



A zone failure in New York fails over to the sync hot standby in seconds with no data loss. Losing all of New York promotes London (RPO seconds, RTO about a minute; it also serves European reads). Tokyo is a cold copy for when both are gone. The etcd quorum spans three sites, so a partitioned minority can never promote itself.

In the interview

30-second answer: "HA = replicas plus automatic failover: heartbeats detect failure, a quorum confirms it to avoid split brain, the most up-to-date standby is promoted, the old primary is fenced, clients are redirected. Sync standbys give zero data loss, async lower latency with some loss. I'd state RPO/RTO, choose hot/warm/cold to match, and test failover regularly."

Self-check

1. *RPO with async replication?* Greater than zero — the lag at the moment of failure.

15.11 Data resilience: redundancy, mirroring, backups

HA keeps the service **running**; **data resilience** makes sure the **data itself** survives disk crashes, corruption, ransomware, operator mistakes and site disasters.

Redundancy

Technique	How	Overhead	Survives
Replication	Full copies on other nodes (15.5)	N×	Node or site loss
RAID 1 / 10	Mirror each disk (10 = mirrors, striped; popular for databases)	2×	1 disk per mirror pair
RAID 5 / 6	Stripe data (spread it across disks) with one / two parity blocks (extra blocks computed from the data, used to rebuild a lost disk)	1 / 2 disks	Any 1 / any 2 disks
Erasure coding	k data + m parity fragments; any k rebuild the data	(k+m)/k: 6+3 → 1.5×	Any m fragments

Erasure coding (e.g. Reed–Solomon) gives replication-level safety at 1.5× instead of 3× storage, paid for with CPU and slower rebuilds — **object stores** (services like Amazon S3 that store whole

files by key) and HDFS (Hadoop's distributed file system) use it for large, cooler (rarely read) data.

Mirroring

Mirroring keeps an **exact, real-time copy**: every write goes to primary and mirror together, and the mirror takes over immediately. Near-instant failover and always current, but double storage and extra write latency. *Example:* a bank mirrors its transaction database synchronously between two nearby data centres. (SQL Server literally called it "Database Mirroring"; Postgres's equivalent is synchronous streaming replication.)

Media failure recovery

A **media failure** is lost or damaged storage — **physical** (dead drive, bad sectors) or **logical** (corruption from software/firmware bugs or malware). Defences: **hot swapping** failed RAID drives while running; **health monitoring** (**SMART** drive telemetry, cloud volume metrics); **data scrubbing** (periodically read every block, verify **checksums** (small fingerprints computed from data; a mismatch means corruption), repair from redundancy — ZFS, Ceph; Postgres detects corrupted pages with `initdb --data-checksums`); and **replication** to other nodes and regions.

Backup and restoration

A **backup** is an independent copy that can take you **back in time** — replicas can't, they follow the primary instantly. **Restoration** gets data back from it.

Type	Contains	Backup cost	Restore needs
Full	Everything	Slowest, largest	Just that backup (simplest restore)
Incremental	Changes since the **last backup of any kind**	Fastest, smallest	Last full + **every** incremental since, in order
Differential	Changes since the **last full**	Grows through the week	Last full + the **latest** differential

PostgreSQL (commands in Chapter 9 — backup and restore): `pg_dump` / `pg_restore` make **logical** backups of one database, portable across versions. `pg_basebackup` plus **continuous WAL archiving** (`archive_mode`, `archive_command`) gives a **physical** backup you can replay to *any moment* — **point-in-time recovery (PITR)** with `recovery_target_time`, the cure for "someone ran `DELETE` without `WHERE` at 14:02". PostgreSQL 17 added incremental base backups (`pg_basebackup --incremental`); pgBackRest and Barman manage all of this.

Best practices: backup frequency to match RPO (daily plus continuous WAL for critical data); **off-site** copies following the **3-2-1 rule** (3 copies, 2 media, 1 off-site); **test restores** regularly; **versioning and immutability** (object lock — stored copies can't be changed or deleted for a set period) so **ransomware** (malware that encrypts or deletes data and demands payment) can't destroy every copy. *Example:* a hospital takes a full backup every Sunday, incrementals Monday–Saturday and continuous WAL, stores copies locally and in another cloud region, and each month restores "last Wednesday 15:00" into a test environment and checks row counts.

> **In the interview**

> **30-second answer:** "Replication and RAID protect against hardware loss but not bad writes, which replicate too. So I add backups — full plus incremental or differential, off-site, versioned, test-restored. In Postgres that's base backups plus WAL archiving for PITR. Erasure coding gives durable storage at about 1.5× instead of 3× for large, cold data."

- *Accidental* `DROP TABLE` *at 14:02?* Restore the base backup into a separate instance, replay WAL to 14:01:59, copy the table back.

Self-check

1. *Which RAID level survives any two disk failures?* RAID 6.
2. *Why isn't a synchronous replica a backup?* It copies mistakes instantly; there's no going back in time.

TL;DR

Redundancy (replicas, RAID, erasure coding) survives hardware loss; backups (full/incremental/differential + WAL archiving) survive mistakes and disasters. Keep copies off-site and versioned, and test restores.

Chapter 15 TL;DR

Topic	Remember
Scale	Scale up first, then scale out with **partitioning + replication** together.
Partitioning	Range (time), list (category), hash (even), composite. Verify pruning with `EXPLAIN`. Unique keys include the partition key.
Sharding	Partitions on separate servers; a high-cardinality shard key matching the main query; consistent hashing with vnodes.
Distributed queries	Push filters and partial aggregates to the data, co-locate joins, semi-join to ship keys not rows.
Replication	Sync = no loss but slower; async = fast, lag, possible loss. Single-leader by default. Leaderless quorums need R + W > N.
Consistency	Strong > sequential > causal > session > eventual. Pick per feature.
CAP / PACELC	Under partition choose C or A; otherwise latency or consistency.
Consensus	Raft/Paxos: 2f+1 nodes survive f crashes. PBFT: 3f+1 for f liars. Elect leaders via etcd/ZooKeeper and fence stale ones.
Recovery	WAL first, checkpoints bound replay, redo committed + undo uncommitted (Postgres: redo-only + MVCC). 2PC blocks, 3PC isn't partition-safe; consensus-backed commit is the modern answer.
HA	Heartbeat → quorum decision → promote → fence → redirect. State RPO/RTO.
Resilience	Replicas are not backups. Full/incremental/differential + PITR, off-site, tested.

Next: Chapter 16 — Beyond relational looks at the NoSQL, NewSQL and specialised databases that make these trade-offs in different ways.